

Eventify

(Smart Event Management and Ticket Booking System)

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of degree of

BACHELOR OF VOCATION (SOFTWARE DEVELOPMENT)

BY

Name: Ibrahim Zummer

UID: [UID]

Under the esteemed guidance of

[Coordinator]

Coordinator

[Internal Guide]

Internal Guide

DEPARTMENT OF SOFTWARE DEVELOPMENT

JAI HIND COLLEGE

(Empowered Autonomous)

MUMBAI, 400020, MAHARASHTRA

2026–27

JAI HIND COLLEGE

(Empowered Autonomous)

MUMBAI, 400020, MAHARASHTRA

DEPARTMENT OF SOFTWARE DEVELOPMENT

CERTIFICATE

This is to certify that the project entitled, **“Eventify — Smart Event Management and Ticket Booking System”**, is the bonafide work of **Ibrahim Zummer**, bearing UID ([UID]), submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF VOCATION (SOFTWARE DEVELOPMENT)** from JAI HIND COLLEGE (Empowered Autonomous) (University of Mumbai).

Internal Guide

Coordinator

Date

External Examiner

College Seal

DECLARATION

I hereby declare that the project entitled **“Eventify — Smart Event Management and Ticket Booking System”**, carried out at JAI HIND COLLEGE, has not in any case been duplicated for submission to any other university for the award of any degree. To the best of my knowledge, other than myself, no one has submitted this work to any other university.

The project has been undertaken in partial fulfilment of the requirements for the award of the degree of BACHELOR OF VOCATION (SOFTWARE DEVELOPMENT), and is submitted as the final semester project forming part of the prescribed curriculum.

All external material referred to during the course of this work — documentation, articles and technical references — has been acknowledged in the Bibliography, and all source code presented in this report was written by me except where explicitly stated otherwise.

Name and Signature of the Student

Ibrahim Zumner

ROLE AND RESPONSIBILITY FORM

This project was designed, developed, tested and documented independently by **Ibrahim Zummer** ([UID]). The table below records the distribution of work across the phases of the project.

Phase / Component	Responsibility	Undertaken By
Requirements gathering and analysis	Problem definition, functional and non-functional requirement specification	Ibrahim Zummer
System design	UML modelling, database schema design, user interface design	Ibrahim Zummer
Frontend development	React component architecture, routing, state management, responsive styling	Ibrahim Zummer
Backend development	Express REST API, controllers, middleware, Mongoose models	Ibrahim Zummer
Authentication and security	JWT issuing and verification, bcrypt hashing, role-based authorisation	Ibrahim Zummer
Payment workflow	Demo gateway, manual UPI submission and administrative confirmation ledger	Ibrahim Zummer
Ticket generation	QR code encoding and client-side PDF ticket export	Ibrahim Zummer
Testing	Unit, integration and system testing; defect logging and resolution	Ibrahim Zummer
Deployment	Frontend deployment on Vercel, backend deployment on Railway, MongoDB Atlas provisioning	Ibrahim Zummer
Documentation	Preparation of this project report	Ibrahim Zummer

Ibrahim Zummer

Internal Guide

ACKNOWLEDGEMENT

I would like to express my sincere thanks to everyone who supported me over the course of this project.

I am grateful to my internal guide, [Internal Guide], for consistent direction and for the patience shown while reviewing successive drafts of both the application and this report. The suggestions offered at each review shaped decisions that materially improved the final system.

I owe my gratitude to [Coordinator], Coordinator of the DEPARTMENT OF SOFTWARE DEVELOPMENT, for taking an active interest in the progress of this work and for providing the guidance and resources needed to see it through to completion.

I would also like to thank the faculty of the DEPARTMENT OF SOFTWARE DEVELOPMENT at JAI HIND COLLEGE for the foundation laid across the programme, without which a project of this scope would not have been approachable.

Finally, I thank my classmates for the discussions that helped clarify several design decisions, and my parents for their encouragement and support throughout.

Ibrahim Zummer

ABSTRACT

Eventify is a web-based event management and ticket booking platform built to replace the spreadsheets, paper registers and scattered messaging that organisations still rely on to run seminars, workshops, cultural programmes and similar gatherings. The platform brings event publishing, ticket reservation, payment recording and administrative oversight into a single application, so that a listing, the tickets sold against it and the money collected for it are all read from one source of truth rather than reconciled by hand after the fact.

The system is built on the MERN stack. A React 18 single-page application, bundled with Vite and styled with Bootstrap 5, provides the interface; an Express server exposes a REST API; and MongoDB Atlas, accessed through the Mongoose ODM, stores the data. Authentication is stateless and token-based: passwords are hashed with bcrypt before storage, JSON Web Tokens carry the caller's identity and role, and every protected route passes through middleware that verifies the token and then checks the role against the operation being attempted.

Three characteristics distinguish the implementation. First, ticket inventory is treated as a resource that must be conserved — a booking decrements the available count, a cancellation restores it, and an event that sells out is moved back to the published state automatically when stock returns. Second, bookings and payments retain a denormalised snapshot of the event as it stood at the moment of purchase, so a subsequent edit to an event listing cannot silently rewrite the history of what a customer actually bought. Third, the platform supports two payment channels side by side: an instant demo gateway, and a manual UPI route in which the customer submits a transfer reference that an administrator reviews and confirms against a payment ledger before the ticket is activated. Confirmed bookings yield a ticket carrying a QR code, which the holder can export as a PDF.

The application was verified through unit, integration and system testing across the authentication, event, booking, payment and administrative modules, and deployed to a public cloud environment with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas. The result is a working demonstration that the operational problems of manual event administration — duplicated records, uncertain ticket availability and untraceable payments — are addressable with a modest, well-structured full-stack application.

Keywords: event management, online ticket booking, MERN stack, React, Express, MongoDB, JSON Web Token, role-based access control, REST API, QR ticketing.

TABLE OF CONTENTS

1. Introduction	11
1.1 Background	11
1.2 Objectives	13
1.3 Purpose, Scope, and Applicability	14
1.3.1 Purpose	14
1.3.2 Scope	14
1.3.3 Applicability	15
1.4 Achievements	17
1.5 Organisation of Report	18
2. Survey of Technologies	19
2.1 Introduction	19
2.2 Front-End Technologies	20
2.2.1 React 18	20
2.2.2 Vite	20
2.2.3 React Router	20
2.2.4 Bootstrap 5	21
2.2.5 HTML5 and CSS3	21
2.2.6 Axios	21
2.2.7 QRCode, jsPDF and html2canvas	22
2.3 Back-End Technologies	23
2.3.1 Node.js	23
2.3.2 Express 5	23
2.3.3 Mongoose	23
2.3.4 JSON Web Token	24
2.3.5 bcrypt	24
2.3.6 Supporting Middleware	24
2.4 Database Technology	25
2.4.1 MongoDB and MongoDB Atlas	25
2.5 Development and Deployment Tools	26
2.5.1 Visual Studio Code	26
2.5.2 Git and GitHub	26

2.5.3	Postman	26
2.5.4	Vercel and Railway	26
2.6	Summary	27
3.	Requirement and Analysis	28
3.1	Problem Definition	28
3.2	Requirements Specification	30
3.2.1	Functional Requirements	30
3.2.2	Non-Functional Requirements	32
3.3	Planning and Scheduling	34
3.3.1	Activity Breakdown	34
3.4	Software and Hardware Requirements	36
3.4.1	Hardware Requirements	36
3.4.2	Software Requirements	36
3.5	Preliminary Product Description	38
3.5.1	Overview	38
3.5.2	Product Features	38
3.5.3	User Characteristics	39
3.5.4	Product Perspective	39
3.5.5	Benefits	39
3.5.6	Limitations	40
3.5.7	Future Enhancements	40
3.6	Conceptual Models	41
3.6.1	Event Table	41
3.6.2	Entity Relationship Diagram	42
3.6.3	Class Diagram	44
3.6.4	Object Diagram	45
3.6.5	Use Case Diagram	45
3.6.6	Activity Diagram	46
3.6.7	Sequence Diagram	47
3.6.8	State Diagram	48
3.6.9	Package Diagram	50
3.6.10	Component Diagram	50
3.6.11	Deployment Diagram	51
3.6.12	Data Flow Diagram	52
4.	System Design	54
4.1	Basic Modules	54
4.1.1	Module 1: User Management and Authentication	54
4.1.2	Module 2: Event Management	55

4.1.3	Module 3: Booking Management	55
4.1.4	Module 4: Payment Management	56
4.1.5	Module 5: Ticketing	56
4.1.6	Module 6: Administration	57
4.1.7	Module 7: Feedback Management (Planned)	57
4.2	Data Design	58
4.2.1	Schema Design	58
4.2.2	Data Integrity and Constraints	62
4.3	User Interface Design	64
4.3.1	Design Principles	64
4.3.2	Input Design	64
4.3.3	Output Design	65
4.3.4	Screen Design	65
4.4	Security Issues	66
4.4.1	Password Storage	66
4.4.2	Authentication	66
4.4.3	Authorisation	66
4.4.4	Configuration and Secret Management	67
4.4.5	Transport and Origin Control	67
4.4.6	Input Handling and Error Disclosure	67
4.4.7	Known Limitations	67
4.5	Test Cases Design	69
5.	Implementation and Testing	72
5.1	Implementation Approach	72
5.1.1	Strategy	72
5.1.2	Architecture	72
5.1.3	Request Lifecycle	73
5.2	Coding Details and Code Efficiency	74
5.2.1	Password Hashing	74
5.2.2	Token Issue and Verification	75
5.2.3	Ticket Inventory Conservation	76
5.2.4	Payment Ledger	78
5.2.5	Token Attachment on the Client	79
5.2.6	Route Guarding on the Client	79
5.2.7	Centralised Error Handling	80
5.2.8	Code Efficiency	81
5.3	Testing Approach	82
5.3.1	Unit Testing	82

5.3.2	Integration Testing	83
5.3.3	System Testing	83
5.4	Modifications and Improvements	85
5.4.1	Outstanding Items	86
6.	Results and Discussion	87
6.1	Test Reports	87
6.1.1	Summary of Results	87
6.1.2	Defects Found and Resolved	87
6.1.3	Discussion	89
6.2	User Documentation	90
6.2.1	Getting Started	90
6.2.2	For Registered Users	90
6.2.3	For Administrators	92
6.2.4	Troubleshooting	93
6.2.5	Interface Reference	94
7.	Conclusion	95
7.1	Conclusion	95
7.1.1	Significance of the System	96
7.2	Limitations of the System	98
7.3	Future Scope of the Project	100
7.3.1	Live Payment Gateway Integration	100
7.3.2	Ticket Validation at the Venue	100
7.3.3	Notification	100
7.3.4	Atomic Inventory Operations	100
7.3.5	Token Revocation and Rate Limiting	101
7.3.6	Search, Filtering and Recommendation	101
7.3.7	Analytics and Reporting	101
7.3.8	Seat Selection	101
7.3.9	Feedback and Rating	101
7.3.10	Media Storage	101
7.3.11	Native Mobile Applications	102
7.3.12	Multi-Language Support	102
	Bibliography	103
	Glossary	107

List of Figures

1	Gantt Chart	35
2	PERT Chart	35
3	Event Table	42
4	ER Diagram	43
5	Class Diagram	44
6	Object Diagram	45
7	Use Case Diagram	46
8	Activity Diagram	47
9	Sequence Diagram	48
10	State Diagram	49
11	Package Diagram	50
12	Component Diagram	51
13	Deployment Diagram	52
14	Data Flow Diagram	53
15	Database Schema Design	58

CHAPTER 1

INTRODUCTION

1.1 Background

Events are a fixture of institutional life. Colleges run seminars, workshops, technical competitions, cultural festivals and sports meets; companies run conferences, training programmes and product launches; community organisations run awareness drives and recreational gatherings. What all of these share is an administrative burden that is almost entirely clerical: someone has to publish what is happening, collect the names of those attending, keep track of how many places remain, take money where a ticket is priced, and produce a record of all of it afterwards.

For a long time this burden was carried by spreadsheets, printed registration forms, counter-sold tickets and a scattering of messages across email and chat groups. The approach works, in the sense that events do get run. It works badly in three specific respects. Records fragment, because the list of registrations, the list of payments received and the list of people who actually attended are maintained separately and reconciled by hand. Availability becomes uncertain, because the number of remaining places is only as accurate as the last manual update, which makes overselling a routine occurrence rather than an exception. And information decays, because a change to a venue or a start time has to be propagated by whoever remembers to propagate it.

Web-based event platforms exist precisely because these are structural problems rather than problems of diligence. Moving the process onto a shared application replaces manual reconciliation with a single stored record that every view reads from. Ticket availability becomes a value the system decrements as part of the same operation that creates a booking, rather than a number someone updates afterwards. Event details are published once and read everywhere. The clerical work does not disappear, but the parts of it that are mechanical — counting, matching, copying — are handed to software, and the parts that require judgement remain with the organiser.

Eventify, the system described in this report, is a web-based event management and ticket booking platform built to demonstrate that shift concretely. It allows any registered user to publish an event, allows other users to browse those events and reserve tickets against them, records the payment associated with each reservation, and gives administrators oversight of the events, bookings and payments on the platform. It is a full-stack JavaScript application:

a React single-page frontend, an Express REST API, and a MongoDB Atlas database, with authentication handled by JSON Web Tokens and passwords protected by bcrypt hashing.

1.2 Objectives

The project was undertaken with the following objectives.

1. To develop a centralised web application through which events can be published, discovered and booked, replacing the manual registers and disconnected communication channels that event administration typically depends on.
2. To implement secure account management, with passwords stored only as bcrypt hashes and sessions carried by signed JSON Web Tokens rather than server-side state.
3. To enforce role-based authorisation, so that the operations available to a registered user and to an administrator are separated at the API level and not merely hidden in the interface.
4. To allow registered users to create, publish, update and withdraw their own events, and to give administrators moderation authority over every event on the platform.
5. To maintain ticket inventory as a conserved quantity, decremented when a booking is confirmed and restored when a booking is cancelled, so that the availability shown to a prospective attendee is always the true availability.
6. To support more than one payment channel — an instant gateway path and a manual UPI path subject to administrative confirmation — and to record every attempt in a payment ledger that an administrator can audit.
7. To issue each confirmed booking a ticket carrying a scannable QR code, exportable by the holder as a PDF document.
8. To provide administrators with consolidated views of the platform's events, bookings, payments and registered users.
9. To design the interface so that it remains usable on desktop, tablet and mobile displays, and to offer both light and dark presentation.
10. To verify the system through unit, integration and system testing, and to deploy the verified application to a public cloud environment.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of Eventify is to give event organisation and ticket booking a single authoritative home. In a manual process the event listing, the register of attendees and the record of payments are three separate artefacts maintained by potentially three different people; the effort of keeping them consistent is what generates most of the administrative overhead, and the failure to keep them consistent is what generates most of the errors. Eventify collapses the three into one stored model in which a booking is structurally attached to the event it is for and the payment that settled it.

The system is therefore intended to serve two purposes at once. For the organiser, it reduces the work of publishing an event and tracking who has booked it to a small number of interface actions, and it removes the possibility of selling more tickets than exist. For the attendee, it turns registration into a self-service operation that can be completed at any hour without correspondence, and it provides a durable, verifiable ticket in return.

A secondary purpose, given the academic context of the work, is to demonstrate the practical construction of a complete full-stack application: component-based frontend architecture, RESTful API design, schema modelling on a document database, stateless authentication, role-based authorisation, and cloud deployment.

1.3.2 Scope

The scope of the system as implemented covers the following.

- **Account management.** Self-service registration, login, and retrieval of the authenticated account, together with a separate administrative login route. Registered users may set a profile image.
- **Event publishing.** Creation of events with a title, description, category, venue, city, organiser name, banner image, start and end date-times, ticket price and total ticket count. Each event carries a lifecycle status — draft, published, sold out or cancelled — and a URL-friendly slug derived from its title.
- **Event discovery.** A public listing of published events, a featured selection surfaced on the landing page, and an individual detail page addressable by slug.
- **Ticket booking.** Reservation of one or more tickets against a published event, validated against remaining availability, producing a booking with a human-readable reference code.

- **Booking management.** A personal booking history for each user, the ability to cancel a booking, and automatic restoration of ticket inventory to the event when a cancellation occurs.
- **Payment handling.** A demo gateway path that settles a paid booking immediately, and a manual UPI path in which the customer submits a transfer reference for administrative review. Every payment attempt is written to a payment ledger.
- **Ticket issue.** Generation of a QR-coded ticket for each confirmed booking, exportable by the holder as a PDF.
- **Administration.** Consolidated listings of all events, bookings, payments and registered users; confirmation of reviewed UPI payments; creation, cancellation and transfer of bookings on a user's behalf; and control over which events are featured.
- **Presentation.** A responsive Bootstrap-based interface with a light and dark theme.

The following lie outside the scope of the present version and are discussed further in Sections 7.2 and 7.3: integration with a live commercial payment gateway, scanning and validation of ticket QR codes at a venue entrance, email and SMS notification, seat-level selection, and analytical reporting beyond the consolidated administrative listings. A feedback and rating module is specified in the design chapters of this report and is scheduled for implementation in a subsequent iteration.

1.3.3 Applicability

The system is applicable wherever events are run on a recurring basis and attendance has to be recorded.

- **Colleges and universities** can publish departmental seminars, workshops, technical competitions and cultural programmes, with attendance recorded per event rather than on paper.
- **Corporate organisations** can administer conferences, internal training sessions and product launches, using the ticket count as a hard capacity control.
- **Event management companies** can list public and private events, take payment through the supported channels, and reconcile receipts against the payment ledger.
- **Non-governmental and community organisations** can run awareness programmes and recreational gatherings, where the free-ticket path removes payment from the process entirely while still producing an attendance record.

Because any registered user may publish an event and administrators moderate rather than originate listings, the platform also suits settings in which event creation is distributed — a college in which individual societies each run their own programme, for instance — rather than centralised in a single administrative office.

1.4 Achievements

The following were completed over the course of the project.

- A complete full-stack web application was built and deployed, comprising a React 18 frontend bundled with Vite, an Express 5 REST API and a MongoDB Atlas database accessed through Mongoose.
- Stateless authentication was implemented end to end: bcrypt password hashing with a ten-round salt applied in a Mongoose pre-save hook, JSON Web Token issue on login, and token verification middleware guarding every protected endpoint.
- Role-based authorisation was implemented as composable route middleware, giving separate capability sets to users and administrators, and a bootstrap routine was added that provisions the administrator account from environment configuration on first start.
- A complete event lifecycle was implemented — draft, published, sold out and cancelled — with slug generation, featured placement with an expiry date, and base64 banner image storage.
- Ticket inventory was implemented as a conserved quantity: bookings decrement availability, cancellations restore it, and an event that had sold out returns to the published state when stock is returned.
- Two payment channels were built: an instant demo gateway and a manual UPI route with administrative confirmation, both writing to a shared payment ledger with reference codes and status tracking.
- Denormalised event snapshots were designed into the booking and payment schemas, so that historical records display the event as it stood at the time of purchase rather than as it stands now.
- QR-coded tickets were implemented, generated client-side and exportable as PDF documents.
- Administrative tooling was built covering event, booking, payment and user listings, UPI confirmation, and booking creation, cancellation and transfer.
- A responsive interface with light and dark themes was implemented and verified across desktop and mobile viewport widths.
- The system was tested at unit, integration and system level, and deployed with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas.

1.5 Organisation of Report

This report is organised into seven chapters, each covering a distinct stage in the development of the system.

Chapter 1 — Introduction establishes the background against which the project was undertaken, states its objectives, defines its purpose, scope and applicability, and records what was achieved.

Chapter 2 — Survey of Technologies reviews the frameworks, libraries, database platform and tooling selected for the implementation, and explains why each was chosen in preference to the alternatives available.

Chapter 3 — Requirement and Analysis defines the problem addressed, specifies the functional and non-functional requirements, presents the project schedule as a Gantt chart and a PERT network, lists the hardware and software required, gives a preliminary description of the product, and presents the conceptual models: the event table and the full set of UML, entity-relationship and data-flow diagrams.

Chapter 4 — System Design describes the modular decomposition of the system, the database schema and the integrity constraints that protect it, the design of the user interface, the security mechanisms adopted, and the design of the test cases used to validate the implementation.

Chapter 5 — Implementation and Testing explains the approach taken to implementation, presents the coding details and the measures taken to keep the code efficient, describes the unit, integration and system testing carried out, and records the modifications and improvements made in response to what testing revealed.

Chapter 6 — Results and Discussion presents the test reports produced during validation and provides user documentation for the completed application.

Chapter 7 — Conclusion draws the work together, states the significance and the limitations of the system as delivered, and identifies the directions in which it could usefully be extended.

CHAPTER 2

SURVEY OF TECHNOLOGIES

2.1 Introduction

Selecting a technology stack for a project of this kind involves a trade-off that is rarely stated explicitly: breadth of capability against the cost of learning and integrating it. A stack assembled from separately-designed parts — a templating language here, an object-relational mapper there, a separate build system for the client — offers more choice at each layer, but each interface between those parts is a place where integration effort accumulates and where failures are hard to diagnose.

The stack chosen for Eventify optimises deliberately for the opposite property: a single language across the whole system. JavaScript runs the browser interface, the API server and the database query layer, which means one syntax, one package manager, one debugging model and one mental model of asynchronous execution throughout. Data crosses every boundary in the system as JSON, from the React component state through the HTTP request body to the BSON document stored by MongoDB, without an intermediate representation to map between. The combination of MongoDB, Express, React and Node.js is conventionally referred to as the MERN stack, and it is the foundation of this project.

This chapter reviews each technology in the stack, describes the role it plays in the system, and gives the reasoning behind its selection.

2.2 Front-End Technologies

2.2.1 React 18

React is an open-source JavaScript library, developed and maintained by Meta, for building user interfaces from composable components. Rather than manipulating the browser's document tree directly, a React application declares what the interface should look like for a given state, and React computes and applies the minimum set of changes required to reach that appearance. It does this by maintaining a lightweight in-memory representation of the tree — the virtual DOM — and diffing successive versions of it.

React was selected for three reasons. Interfaces of this kind are naturally component-shaped: an event card, a booking ticket and an authentication form each have their own markup, styling and behaviour, and each recurs across several screens. React's component model expresses that reuse directly, and the project's frontend is accordingly structured as a set of reusable components composed into page-level views. Second, React's Context API provides state sharing across the component tree without threading values through intermediate components; the project uses it for the two concerns that are genuinely global — the authenticated session and the active theme. Third, the volume of documentation, community answers and third-party libraries available for React is larger than for any comparable option, which materially reduces the time cost of an unfamiliar problem.

2.2.2 Vite

Vite is the build tool and development server used for the frontend. During development it serves source files as native ES modules directly to the browser rather than bundling them first, which means the development server starts in a fraction of a second regardless of project size and applies edits to the running page almost immediately through hot module replacement. For production it builds an optimised, minified and code-split bundle using Rollup.

Vite was chosen over the older Create React App tooling because the development feedback loop is substantially faster, because it is actively maintained where Create React App is not, and because its production build supports dynamic imports cleanly — a property the project relies on to load the QR code and PDF export libraries only when a user actually requests a ticket, rather than including them in the initial bundle delivered to every visitor.

2.2.3 React Router

React Router provides client-side routing for the single-page application, mapping URL paths to the components that should render for them and updating the address bar without a

full page reload. The project uses it for the complete route table — the public home, events and event-detail pages; the authenticated dashboard, bookings and my-events pages; and the administrative event, booking and payment pages — and, importantly, for route guarding. Two wrapper components, `ProtectedRoute` and `PublicOnlyRoute`, sit between the router and the page components and redirect based on authentication state and role, so that an unauthenticated visitor cannot reach a dashboard and an authenticated user is not shown the login form.

2.2.4 Bootstrap 5

Bootstrap is the CSS framework used for layout and visual styling. It supplies a twelve-column responsive grid, a set of pre-built components — navigation bars, cards, forms, tables, modals, badges — and a system of utility classes for spacing, typography and colour.

The argument for using a framework here rather than writing stylesheets from scratch is one of scope control. Responsive behaviour that works correctly across phone, tablet and desktop viewports is a substantial piece of work in its own right, and it is not the piece of work this project set out to demonstrate. Bootstrap provides that behaviour as a starting point, and the project layers a custom stylesheet over it to carry the application's own visual identity and its light and dark themes. The companion Bootstrap Icons package supplies the interface iconography.

2.2.5 HTML5 and CSS3

HTML5 provides the document structure underneath React's rendering, contributing semantic elements that improve both accessibility and the readability of the generated markup, and native form validation attributes that give immediate client-side feedback before a submission reaches JavaScript. CSS3 supplies the styling layer: custom properties, which the project uses to implement theme switching by redefining a palette rather than duplicating rules; Flexbox and Grid for layout; media queries for responsive breakpoints; and transitions for interface feedback.

2.2.6 Axios

Axios is the HTTP client used for all communication between the frontend and the API. It was preferred over the browser's native `fetch` function chiefly for its interceptors: the project configures a single Axios instance with a request interceptor that attaches the stored JSON Web Token as an `Authorization` header on every outbound call. Without that mechanism, token attachment would have to be repeated at every call site. Axios also parses JSON responses automatically and treats HTTP error statuses as rejected promises,

which `fetch` does not.

2.2.7 QRCode, jsPDF and html2canvas

These three libraries together produce the downloadable ticket. The `qrcode` library encodes a booking's reference data into a QR code rendered as a data URL, which is embedded in the on-screen ticket component. When the holder requests a copy, `html2canvas` rasterises the rendered ticket element to a canvas and `jsPDF` places that image into a PDF document sized to fit it.

Generating the ticket in the browser rather than on the server was a deliberate choice: it removes an entire class of server-side rendering dependency, keeps the ticket visually identical to what the user sees on screen, and costs the server nothing. All three libraries are loaded through dynamic imports, so a visitor who never downloads a ticket never downloads the code that would have produced one.

2.3 Back-End Technologies

2.3.1 Node.js

Node.js is the server-side JavaScript runtime on which the API executes. It is built on the V8 engine and uses an event-driven, non-blocking I/O model: rather than dedicating a thread to each connection and blocking that thread while a database query completes, Node.js registers a callback and continues processing other requests until the result arrives.

That model suits this application closely. Almost everything the API does is I/O bound — reading and writing MongoDB documents, hashing and comparing passwords, signing and verifying tokens — and very little of it is computationally expensive. Under those conditions the non-blocking model sustains a considerably higher number of concurrent requests per unit of memory than a thread-per-request server would. The additional benefit, already noted, is language continuity with the frontend.

2.3.2 Express 5

Express is the web application framework layered over Node.js. It provides HTTP routing, a middleware pipeline, and request and response abstractions, without imposing an opinion about project structure.

The middleware pipeline is the feature the project depends on most heavily. Cross-cutting concerns are each expressed as a small function that receives the request, does one thing, and passes control onward: CORS negotiation, JSON body parsing with an increased size limit to accommodate base64 banner images, HTTP request logging, token verification, role authorisation, not-found handling and centralised error formatting. Because these compose by ordering rather than by inheritance, a protected administrative route is written simply as the route path followed by the token middleware, the role middleware and the controller. Express 5, the version used, is the first release to propagate errors from rejected promises in asynchronous handlers automatically.

2.3.3 Mongoose

Mongoose is the Object Data Modelling library through which the application communicates with MongoDB. MongoDB itself imposes no structure on the documents in a collection; Mongoose adds a schema layer above it that declares the fields a document may contain, their types, whether they are required, and what values they may take.

This matters for an application that handles money and inventory. Without a schema layer, a typographical error in a status string is simply stored, and the resulting invalid state is

discovered later at the point where something reads it. With Mongoose, the enumerated validators on the event, booking and payment status fields reject the write outright. Mongoose also supplies the middleware hooks used to hash passwords transparently before persistence, the instance methods used to compare a candidate password and to strip the password hash from any document sent to a client, the population mechanism for resolving references, and automatic `createdAt` and `updatedAt` timestamps.

2.3.4 JSON Web Token

JSON Web Tokens provide the authentication mechanism. On a successful login the server signs a token containing the user's identifier and role using a secret held in environment configuration; the client stores that token and presents it on subsequent requests, and the server verifies the signature to establish who is calling.

The property that makes this approach attractive is statelessness. The server holds no session store, so any instance of the API can serve any request without shared session state — which is what allows the backend to be deployed and scaled independently of the frontend. The corresponding trade-off, acknowledged in Section 4.4, is that a token cannot be revoked before it expires without reintroducing server-side state.

2.3.5 bcrypt

bcrypt is the password hashing function used to protect stored credentials. It is deliberately slow, and its cost factor is configurable, which is precisely the property required: an attacker who obtains the user collection must spend that cost on every candidate password they test. The project applies ten salt rounds, and bcrypt incorporates a distinct random salt into each hash, so identical passwords produce different stored values and precomputed rainbow tables offer no advantage. Verification uses bcrypt's own constant-time comparison rather than string equality, which avoids leaking information through response timing.

2.3.6 Supporting Middleware

`cors` configures Cross-Origin Resource Sharing so that the API accepts browser requests only from the configured frontend origin. `dotenv` loads configuration — the database connection string, the token secret, the administrator bootstrap credentials — from an environment file that is excluded from version control. `morgan` logs HTTP requests during development. `nodemon` restarts the development server automatically when a source file changes.

2.4 Database Technology

2.4.1 MongoDB and MongoDB Atlas

MongoDB is a document-oriented NoSQL database that stores records as BSON documents — a binary encoding of a JSON-like structure — grouped into collections. MongoDB Atlas is the managed cloud service that runs it, handling provisioning, replication, backup and network access control.

A document database was chosen over a relational one for reasons specific to this application’s data. The entities involved — users, events, bookings and payments — have flat structures with few genuine many-to-many relationships, so the normalisation that a relational schema provides buys comparatively little. Against that, the application positively benefits from the ability to store a snapshot of an event inside a booking document: because a booking records the event’s title, slug, start date, venue and city as they stood at the moment of purchase, the booking remains a truthful record of what was sold even after the event listing is subsequently edited. Expressing that in a normalised relational schema would require either duplicated columns or a versioned event table. Storing it as fields on the booking document is the natural expression of the same idea.

The operational argument is equally direct. Atlas provides a free tier adequate for a project of this scale, removes database administration from the project’s scope entirely, and exposes the cluster over an authenticated connection string that lives in environment configuration rather than in the codebase.

2.5 Development and Deployment Tools

2.5.1 Visual Studio Code

Visual Studio Code was the editor used throughout. Its JavaScript language support, integrated terminal, integrated Git interface, debugger and extension ecosystem cover the project's needs without additional configuration.

2.5.2 Git and GitHub

Git provided version control. Development proceeded through a sequence of incremental commits, and the resulting history — which runs from the initial build-out through the payment gateway work to the final patches — is used in Section 5.4 as the record of how the system evolved. GitHub hosted the remote repository, provided off-machine backup, and served as the deployment source for both hosting platforms.

2.5.3 Postman

Postman was used to exercise the REST API independently of the frontend. Each endpoint was tested through Postman first — valid input, invalid input, missing token, wrong role — before any interface was built against it, which isolates API defects from interface defects and makes the source of a failure unambiguous.

2.5.4 Vercel and Railway

Vercel hosts the frontend. It builds directly from the GitHub repository, serves the resulting static bundle from a global edge network, and provisions HTTPS automatically. The project's `vercel.json` configures a rewrite that directs all paths to `index.html`, which is what allows client-side routing to work on a deep link or a page refresh.

Railway hosts the backend. It builds the Node.js service from the same repository, injects the environment configuration, and exposes the API over HTTPS. Splitting the deployment this way — static assets on a content delivery network, the API on an application host — suits the architecture: the frontend is genuinely static once built, and the two tiers scale on entirely different criteria.

2.6 Summary

The technologies described in this chapter form a single-language stack in which JavaScript spans the interface, the API and the data layer, and JSON is the data representation at every boundary. React with Vite provides a fast, component-structured frontend; Express on Node.js provides a middleware-composed REST API; Mongoose over MongoDB Atlas provides schema-validated document storage; and JSON Web Tokens with bcrypt provide stateless authentication over securely hashed credentials. Each was selected for a property the application actually relies on rather than for general popularity, and the combination proved sufficient to implement the full scope described in Section 1.3.2 without recourse to additional frameworks.

CHAPTER 3

REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Event administration in most institutions is not a single process but a loose collection of habits. A notice announces the event. A spreadsheet, a paper register or a messaging group collects the names of those who want to attend. Money, where money is involved, changes hands separately and is recorded by whoever collected it. On the day, someone checks arrivals against whichever list they happen to be holding. Each of these steps works in isolation. The difficulty is that they do not connect, and the work of connecting them falls on people.

Four specific failures follow from that disconnection.

Records fragment and disagree. Because registration, payment and attendance are recorded separately, reconciling them is a manual comparison performed after the fact. When the lists disagree — a name on the register with no matching payment, a payment with no matching name — there is often no way to establish which is correct, because neither record carries enough context to settle the question.

Availability is unreliable. The number of places remaining is a figure someone maintains by hand. Between the moment a registration is accepted and the moment the count is updated, the published availability is wrong. When two registrations arrive close together, or when several people are accepting registrations at once, the count drifts, and overselling becomes a routine occurrence rather than an exception.

Information decays. A change to a venue, a start time or a ticket price has to be propagated by hand to every place it was published. Whichever copy is not updated becomes a source of incorrect information that continues to circulate, and attendees who acted on it arrive at the wrong place or the wrong time.

Payment is untraceable. A payment collected in cash or by direct transfer leaves no structured record tying it to a specific person and a specific event. Verifying that a given attendee has paid becomes an exercise in memory or in searching through transaction messages, and refunding a cancellation is correspondingly awkward.

These are not failures of care. They are structural consequences of holding related information in unrelated places. Solving them requires a system in which the registration, the ticket

count and the payment are parts of one stored record rather than three separate artefacts that a person is expected to keep aligned.

The problem this project addresses is therefore the design and implementation of a web-based platform in which an event, the tickets sold against it and the payments received for it are structurally connected; in which ticket availability is decremented by the same operation that creates a booking, so that it cannot drift; in which event details are published once and read everywhere; and in which every payment attempt is written to an auditable ledger tied to the booking it settles.

3.2 Requirements Specification

3.2.1 Functional Requirements

The functional requirements describe what the system is required to do. They are grouped by the module responsible for them.

ID	Module	Requirement
FR-01	User Management	The system shall allow a visitor to register an account by supplying a name, an email address and a password.
FR-02	User Management	The system shall reject a registration whose email address is already associated with an existing account.
FR-03	User Management	The system shall authenticate a registered user against a stored password hash and issue a signed access token on success.
FR-04	User Management	The system shall provide a separate authentication route for administrators.
FR-05	User Management	The system shall return the profile of the currently authenticated account, and shall allow that account to update its profile image.
FR-06	Event Management	The system shall allow an authenticated user to create an event with a title, description, category, venue, city, organiser name, banner image, start and end date-time, ticket price and total ticket count.
FR-07	Event Management	The system shall generate a unique URL-friendly slug for each event from its title.
FR-08	Event Management	The system shall allow the owner of an event, and any administrator, to update or delete that event.
FR-09	Event Management	The system shall publish a list of events whose status is published, and shall serve an individual event by its slug.
FR-10	Event Management	The system shall allow an administrator to designate an event as featured, with an expiry date, and to withdraw that designation.
FR-11	Booking Management	The system shall allow an authenticated user to book one or more tickets against a published event.

ID	Module	Requirement
FR-12	Booking Management	The system shall reject a booking whose requested quantity exceeds the tickets currently available.
FR-13	Booking Management	The system shall decrement the event's available ticket count when a booking is created, and set the event's status to sold out when the count reaches zero.
FR-14	Booking Management	The system shall assign each booking a unique human-readable reference code.
FR-15	Booking Management	The system shall present each user with a history of their own bookings, and shall allow a user to cancel a booking.
FR-16	Booking Management	The system shall restore the cancelled quantity to the event's available ticket count on cancellation, returning a sold-out event to the published state.
FR-17	Payment Management	The system shall settle a zero-priced booking without requiring payment.
FR-18	Payment Management	The system shall provide a demo gateway path that confirms a paid booking immediately.
FR-19	Payment Management	The system shall provide a manual UPI path in which the user submits a transfer reference for administrative review.
FR-20	Payment Management	The system shall allow an administrator to confirm a reviewed UPI payment, which activates the associated ticket.
FR-21	Payment Management	The system shall record every payment attempt in a payment ledger with a unique reference, provider, method, amount, currency and status.
FR-22	Ticketing	The system shall generate a QR code for each confirmed booking and allow the holder to export the ticket as a PDF document.
FR-23	Administration	The system shall provide administrators with consolidated listings of all events, bookings, payments and registered users.
FR-24	Administration	The system shall allow an administrator to create, cancel and transfer a booking on a user's behalf.

ID	Module	Requirement
FR-25	Administration	The system shall expose a health endpoint reporting API and database availability.

A Feedback Management requirement set — submission of a rating and comment against an attended event, and administrative review of submitted feedback — is carried through the conceptual models in Section 3.6 and the modular design in Section 4.1. It is scheduled for the next development iteration and is not present in the implementation described in Chapter 5.

3.2.2 Non-Functional Requirements

ID	Attribute	Requirement
NFR-01	Security	Passwords shall never be stored or transmitted in plain text; they shall be persisted only as bcrypt hashes.
NFR-02	Security	Every endpoint that reads or modifies account-specific data shall require a valid access token, and shall additionally verify the caller's role where the operation is role-restricted.
NFR-03	Security	Configuration secrets — the database connection string, the token signing secret and the administrator bootstrap credentials — shall be supplied through environment variables and excluded from version control.
NFR-04	Security	The API shall accept browser requests only from the configured frontend origin.
NFR-05	Performance	Interactive pages shall render within approximately two seconds on a broadband connection, and libraries used only for ticket export shall be loaded on demand rather than in the initial bundle.
NFR-06	Reliability	Ticket availability shall remain consistent with the set of active bookings at all times, including after cancellations.
NFR-07	Reliability	Historical bookings and payments shall continue to display the event details in force at the time of purchase, irrespective of later edits to the event.

ID	Attribute	Requirement
NFR-08	Usability	The interface shall be operable on desktop, tablet and mobile viewport widths without horizontal scrolling, and shall offer a light and a dark theme.
NFR-09	Usability	Every rejected input shall produce a specific, human-readable message identifying what was wrong.
NFR-10	Maintainability	The codebase shall separate routing, request handling, data modelling and cross-cutting middle-ware into distinct layers.
NFR-11	Portability	The application shall run on any platform supporting a current Node.js runtime and a modern browser, with no client-side installation required.
NFR-12	Scalability	The API shall hold no server-side session state, so that additional instances can serve requests without shared session storage.

3.3 Planning and Scheduling

Development was planned in two complementary views. The PERT network in Figure 2 expresses the *logical* plan — which activities depend on which, and where the schedule has no slack. The Gantt chart in Figure 1 expresses the *calendar* plan actually followed, in which several activities were deliberately overlapped rather than run strictly in sequence.

3.3.1 Activity Breakdown

ID	Activity	Duration (days)	Depends on
A	Requirement Gathering	3	—
B	System Analysis	2	A
C	Database Design	3	B
D	User Interface Design	4	B
E	Frontend Development	8	D
F	Backend Development	8	C
G	Integration	4	E, F
H	Testing	4	G
I	Deployment and Documenta- tion	3	H

Applying the forward and backward pass to this network gives a project duration of 28 days along the critical path $A \rightarrow B \rightarrow D \rightarrow E \rightarrow G \rightarrow H \rightarrow I$. Database Design (C) and Backend Development (F) each carry one day of slack: the design work on the data model could begin, and the corresponding backend work could finish, one day later than planned without delaying integration. Every other activity is critical, which is the expected shape for a single-developer project — with no second developer to absorb work in parallel, almost every task lies directly on the path.

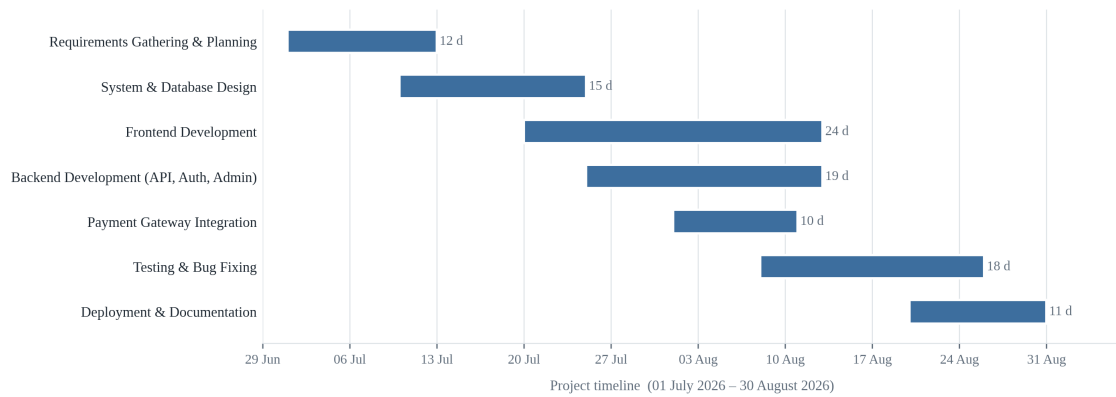


Fig. 1: Gantt Chart

The Gantt chart records the calendar schedule as executed, across the window from 01 July 2026 to 30 August 2026. Three overlaps are visible and were intentional. Frontend development began before system and database design was complete, because the interface work for authentication and event browsing did not depend on the final shape of the payment schema. Backend development ran concurrently with frontend development for nineteen days, with the API for each feature built immediately before the interface that consumed it. Testing began while payment gateway integration was still in progress, since the authentication, event and booking modules were by then stable enough to test in earnest. The dates align with the project’s version control history, in which the initial build-out is recorded from 31 July 2026, the payment gateway work from 05–06 August 2026, and iterative patches through 25 August 2026.

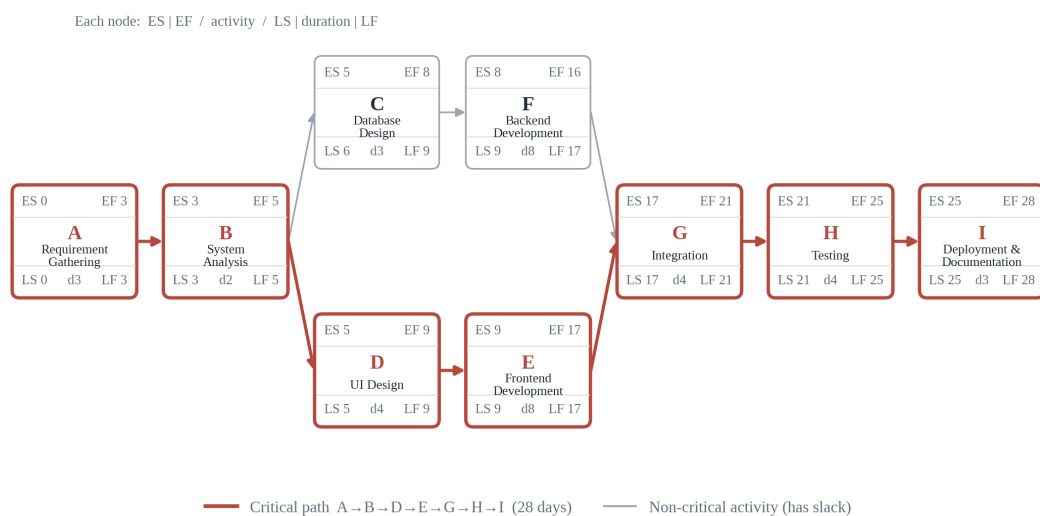


Fig. 2: PERT Chart

3.4 Software and Hardware Requirements

3.4.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (8th generation) or AMD equivalent	Intel Core i5/i7, AMD Ryzen 5 or Apple M-series
Memory	4 GB RAM	8 GB RAM or higher
Storage	20 GB free disk space	40 GB free on solid-state storage
Display	1366 × 768	1920 × 1080
Network	Broadband connection	Stable broadband connection

These figures describe a development machine. The deployed application imposes no such requirement on its users: because the frontend is a static bundle served over HTTPS and the API runs on a remote host, an end user needs only a current web browser and a network connection, on a desktop computer, tablet or phone.

3.4.2 Software Requirements

Software	Version	Purpose
Windows 10/11, macOS or Linux	—	Development operating system
Node.js	18 LTS or later	JavaScript runtime for the API and the build tooling
npm	9 or later	Dependency management
Visual Studio Code	Current	Source code editor
Git	2.x	Version control
React	18.3	Frontend user interface library
Vite	7.1	Frontend build tool and development server
React Router DOM	6.30	Client-side routing
Bootstrap	5.3	Responsive CSS framework
Axios	1.11	HTTP client
qrcode / jsPDF / html2canvas	1.5 / 4.2 / 1.4	QR encoding and PDF ticket export
Express	5.1	Backend web framework

Software	Version	Purpose
Mongoose	8.18	Object data modelling for MongoDB
jsonwebtoken	9.0	Access token issue and verification
bcrypt	6.0	Password hashing
cors / dotenv / morgan	2.8 / 17.2 / 1.10	Origin control, configuration, request logging
nodemon	3.1	Development server auto-restart
MongoDB Atlas	Cloud (M0)	Managed database cluster
Postman	Current	REST API testing
StarUML	Current	UML, ER and data-flow modelling
Google Chrome / Microsoft Edge	Current	Target browsers
Vercel / Railway	Cloud	Frontend and backend hosting

3.5 Preliminary Product Description

3.5.1 Overview

Eventify is a web application that brings event publication, ticket booking, payment recording and administrative oversight into a single platform. A registered user may publish an event, browse events published by others, reserve tickets against them, settle the associated payment through one of the supported channels, and hold the resulting QR-coded ticket. An administrator has oversight of the whole platform: every event, every booking, every payment and every registered account.

The application is reached through a web browser and requires no installation. It presents a public face — a landing page with featured events, an events listing and individual event pages — that is available without an account, and an authenticated area whose contents depend on the role of the account signed in.

3.5.2 Product Features

- Account registration and authentication, with a separate administrative sign-in.
- Event creation, editing, withdrawal and lifecycle management across the draft, published, sold-out and cancelled states.
- Public event discovery with a featured selection and individual event pages addressed by slug.
- Ticket booking validated against live availability, with automatic decrement of the available count and automatic transition to sold out.
- Booking history, cancellation, and restoration of ticket inventory on cancel.
- Free, demo-gateway and manual-UPI payment channels, all recorded in an auditable payment ledger.
- Administrative confirmation of reviewed UPI payments.
- QR-coded tickets exportable as PDF documents.
- Consolidated administrative listings for events, bookings, payments and users, with the ability to create, cancel and transfer bookings on a user's behalf.
- A responsive interface with light and dark themes.

3.5.3 User Characteristics

The system recognises two roles.

Registered user. A person with an account. They may browse and search events, view event details, publish and manage their own events, book tickets, complete payment, view and cancel their bookings, download tickets, and maintain a profile image. Their authority is limited to their own events and their own bookings.

Administrator. A privileged account provisioned from environment configuration when the server first starts. Administrators see every event, booking, payment and registered account on the platform. They may edit or delete any event, control which events are featured, confirm reviewed UPI payments, and create, cancel or transfer bookings on behalf of users. Notably, administrators moderate rather than originate: event creation is a user capability, which reflects a platform in which listings come from the community and administration exists to oversee them.

No technical knowledge is assumed of either role. Both interact solely through the web interface.

3.5.4 Product Perspective

Eventify is a self-contained web application built on a three-tier client–server architecture. The presentation tier is a React single-page application served as a static bundle from a content delivery network. The application tier is an Express REST API that holds all business logic — validation, authorisation, inventory arithmetic and payment state transitions. The data tier is a MongoDB Atlas cluster accessed exclusively by the API through Mongoose.

The tiers communicate over HTTPS using JSON. Because the API holds no session state, identity travels with each request in the form of a signed token, and the application tier can be scaled or replaced without affecting the other two. The system has no dependency on any external service beyond its hosting platforms and its database; the payment channels are handled internally rather than delegated to a commercial gateway.

3.5.5 Benefits

- Event details, bookings and payments are held as one connected record, removing the manual reconciliation that fragmented records require.

- Ticket availability is maintained by the system rather than by a person, so overselling is structurally prevented rather than merely discouraged.
- Every payment is recorded against a specific booking, a specific user and a specific event, making verification and refund straightforward.
- Bookings and payments preserve the event as it stood at purchase, so the record of what a customer bought cannot be altered by a later edit.
- Event publication is distributed to users rather than routed through a central administrator, removing a bottleneck.
- Attendees receive a durable, verifiable ticket rather than a message or a name on a list.
- The platform is reachable from any browser on any device, with no installation.

3.5.6 Limitations

- An internet connection is required; the application has no offline mode.
- Payment is settled through an internal demo gateway or a manually reviewed UPI transfer rather than through a live commercial payment gateway.
- Ticket QR codes are generated and displayed but are not yet validated by a scanning application at a venue entrance.
- Email and SMS notification are not implemented.
- Administrative insight is limited to consolidated listings; aggregated analytics and reporting are not provided.
- Feedback and rating functionality is specified but not yet implemented.

3.5.7 Future Enhancements

Integration with a live payment gateway, a companion scanning application for ticket validation at entry, email and SMS notification, seat-level selection for seated venues, an analytics dashboard, native mobile applications, and multi-language support. These are discussed in detail in Section 7.3.

3.6 Conceptual Models

This section presents the conceptual models produced during analysis and design. Together they describe the system from complementary angles: what happens and when (event table, activity, sequence and state diagrams), what the system is made of (use case, class, object, package, component and deployment diagrams), what data it holds (entity relationship diagram) and how data moves through it (data flow diagrams).

3.6.1 Event Table

The event table enumerates the events the system responds to, the trigger that raises each one, the processing it performs, and the response returned.

Event	Trigger	Action Performed	System Response
Register	Visitor submits the registration form	Validates the input, rejects a duplicate email, hashes the password and stores the account	Account created and the user signed in
Login	User submits email and password	Compares the password against the stored hash and issues a signed token	Dashboard displayed
Admin Login	Administrator submits credentials on the hidden admin route	Authenticates and verifies the administrator role	Administrative dashboard displayed
View Events	User opens the events page	Retrieves all events whose status is published	Event list displayed
View Event Details	User selects an event	Retrieves the event by its slug	Event detail page displayed
Create Event	User submits the event form	Validates the input, generates a unique slug and stores the event	Event created and listed under the user's events
Update Event	Owner or administrator edits an event	Verifies authority and applies the changes	Event updated
Delete Event	Owner or administrator removes an event	Verifies authority and deletes the record	Event removed from the listing

Event	Trigger	Action Performed	System Response
Book Ticket	User confirms a booking	Checks availability, creates the booking, assigns a reference and decrements the available count	Booking confirmed and reference issued
Make Payment	User submits payment through the demo gateway or the UPI form	Records the payment, and for the demo path confirms the booking immediately	Payment recorded and ticket activated or held for review
Confirm UPI Payment	Administrator approves a reviewed transfer	Marks the payment paid and confirms the associated booking	Ticket activated
View Bookings	User opens their bookings page	Retrieves the bookings belonging to that user	Booking history displayed
Download Ticket	User requests a ticket copy	Encodes the booking reference as a QR code and renders the ticket to a PDF	Ticket file downloaded
Cancel Booking	User cancels a booking	Marks the booking cancelled, marks its payment refunded and returns the tickets to the event	Booking cancelled and inventory restored
Manage Events	Administrator opens the events console	Retrieves every event regardless of status	Full event list displayed
Manage Bookings	Administrator opens the bookings console	Retrieves every booking on the platform	Full booking list displayed
View Payments	Administrator opens the payments console	Retrieves the complete payment ledger	Payment ledger displayed
Manage Users	Administrator opens the users console	Retrieves every registered account	User list displayed
Logout	User or administrator signs out	Discards the stored access token	Session ended and the user returned to the public site

Fig. 3: Event Table

3.6.2 Entity Relationship Diagram

The entity relationship diagram models the persistent data as entities and the relationships between them. USER is the root entity: a user creates events and places bookings. Each

BOOKING references the USER who made it and the EVENT it is for, and each PAYMENT references the BOOKING it settles. FEEDBACK, shown here as part of the designed model, associates a rating and comment with a user and an event.

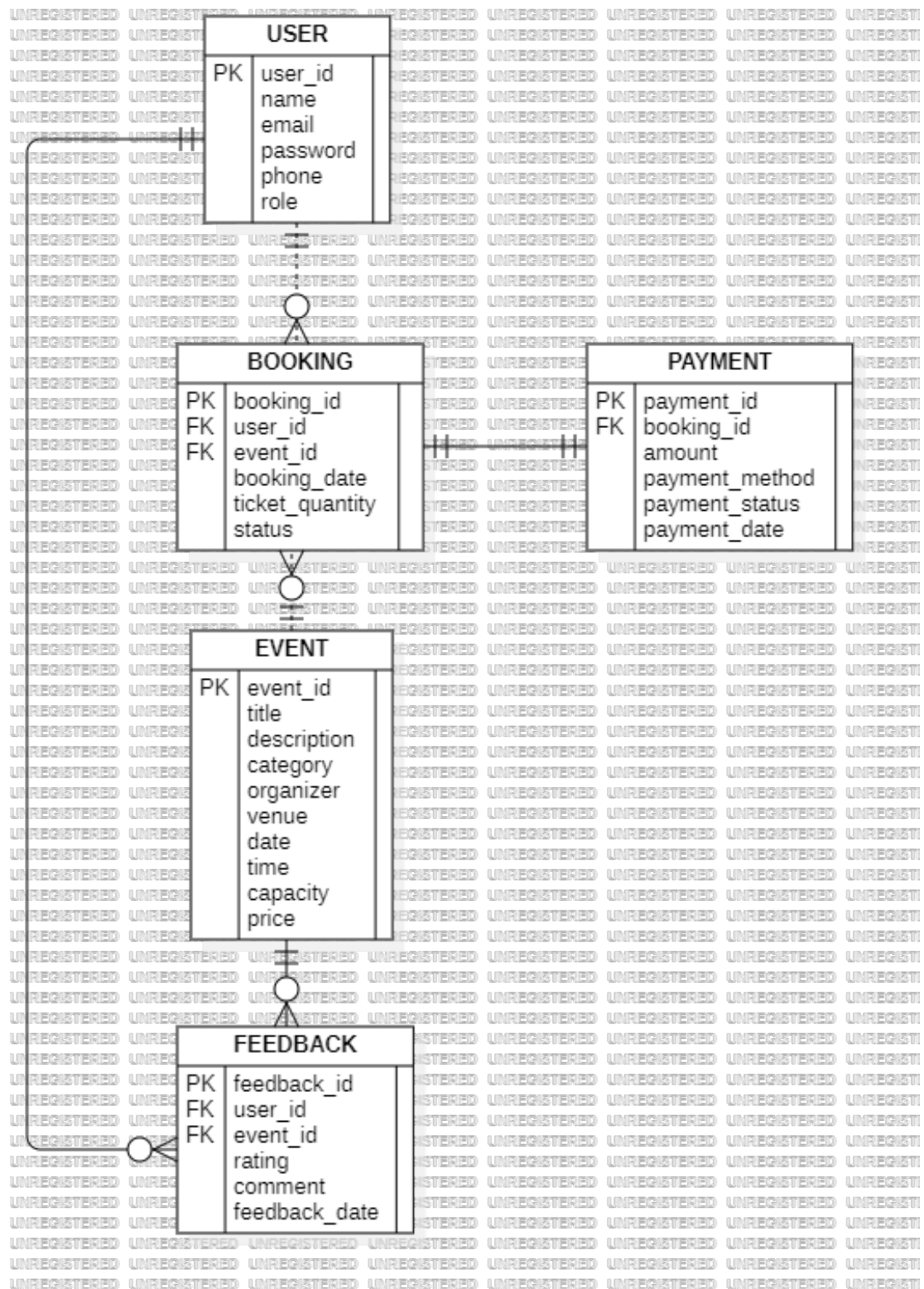


Fig. 4: ER Diagram

3.6.3 Class Diagram

The class diagram presents the same entities as software classes, with their attributes and the operations defined on them. It shows, for example, that User is responsible for registration, login, logout and profile maintenance, while Booking owns the operations for creating, cancelling and retrieving a reservation.

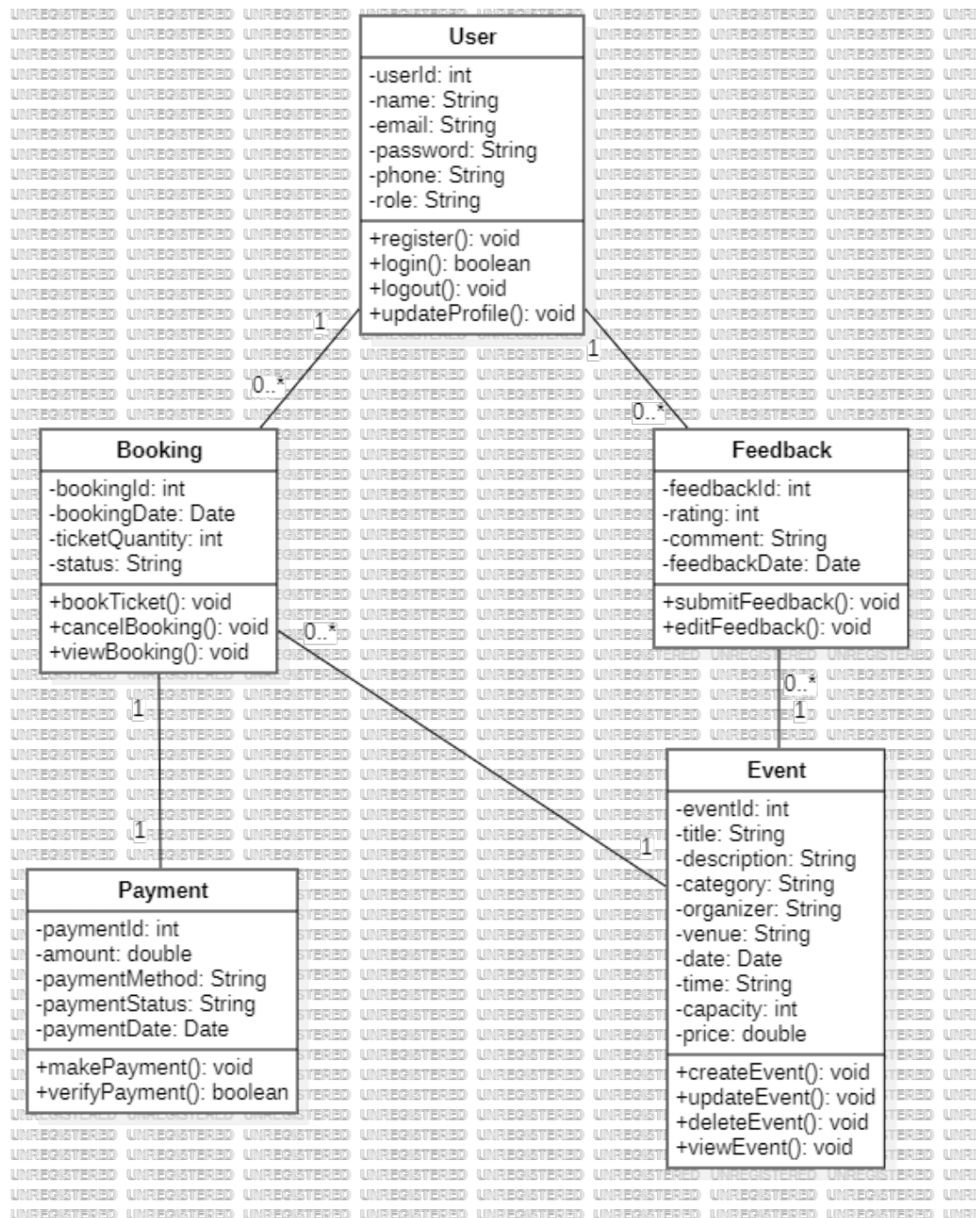


Fig. 5: Class Diagram

3.6.4 Object Diagram

The object diagram shows a snapshot of the system at a single moment: concrete instances of the classes above, populated with representative values and linked as they would be at runtime. It makes the relationships in the class diagram concrete — one specific user holding one specific booking against one specific event.

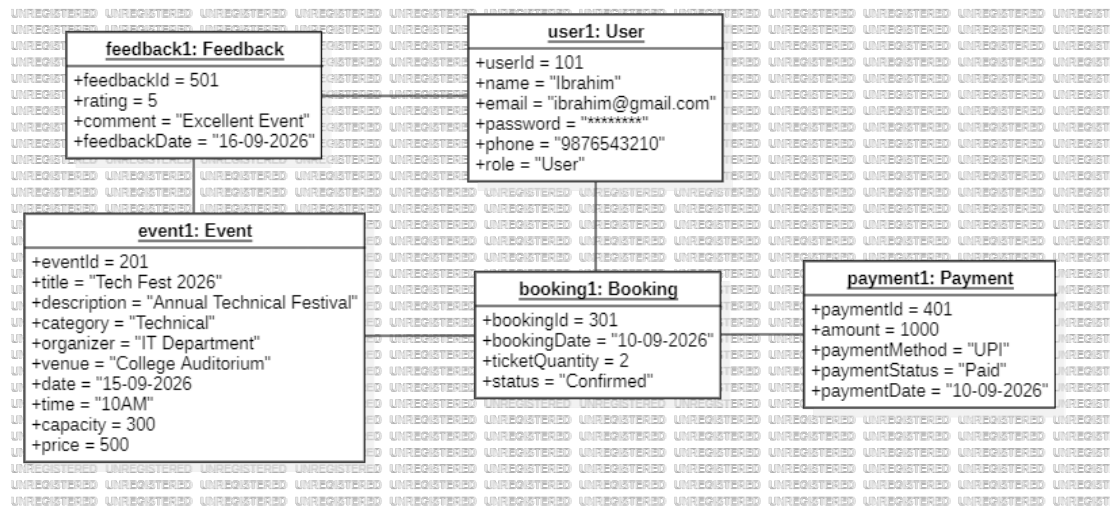


Fig. 6: Object Diagram

3.6.5 Use Case Diagram

The use case diagram identifies the two actors — User and Admin — and the use cases each can invoke. The User actor registers, signs in, browses and books events, makes payments and manages a profile. The Admin actor manages users, events and bookings, and reviews payments.

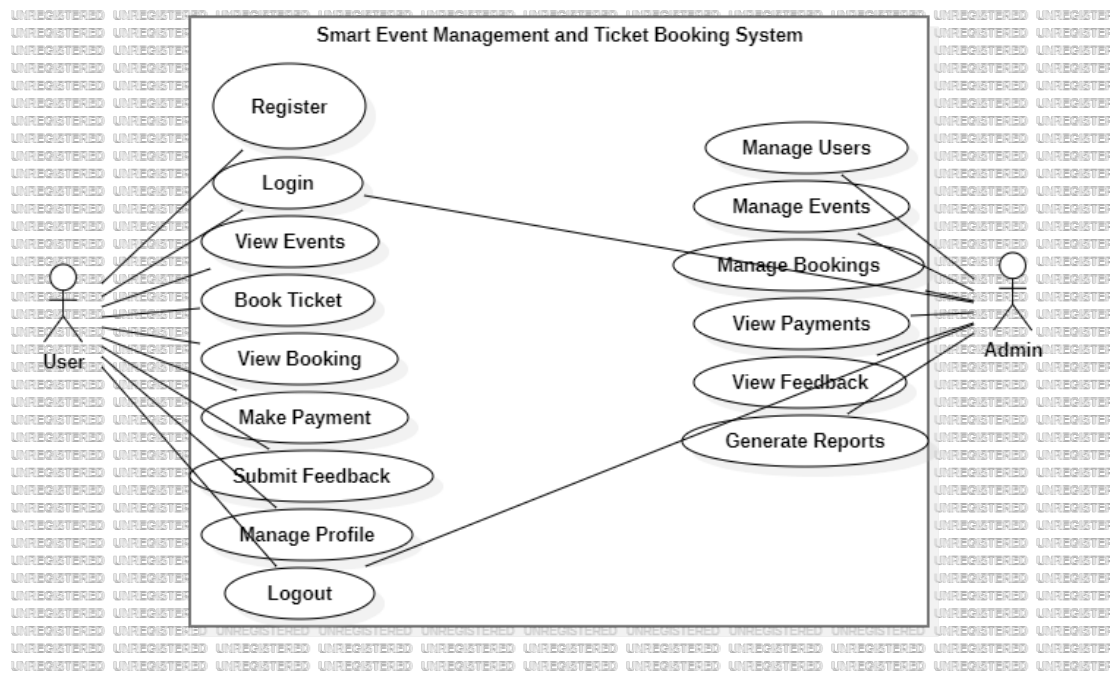


Fig. 7: Use Case Diagram

3.6.6 Activity Diagram

The activity diagram traces the principal user journey from arrival to sign-out, including the two decision points that govern it: whether authentication succeeded, and whether payment succeeded. A failed authentication returns the visitor to the sign-in form; a failed payment offers a retry rather than discarding the booking.

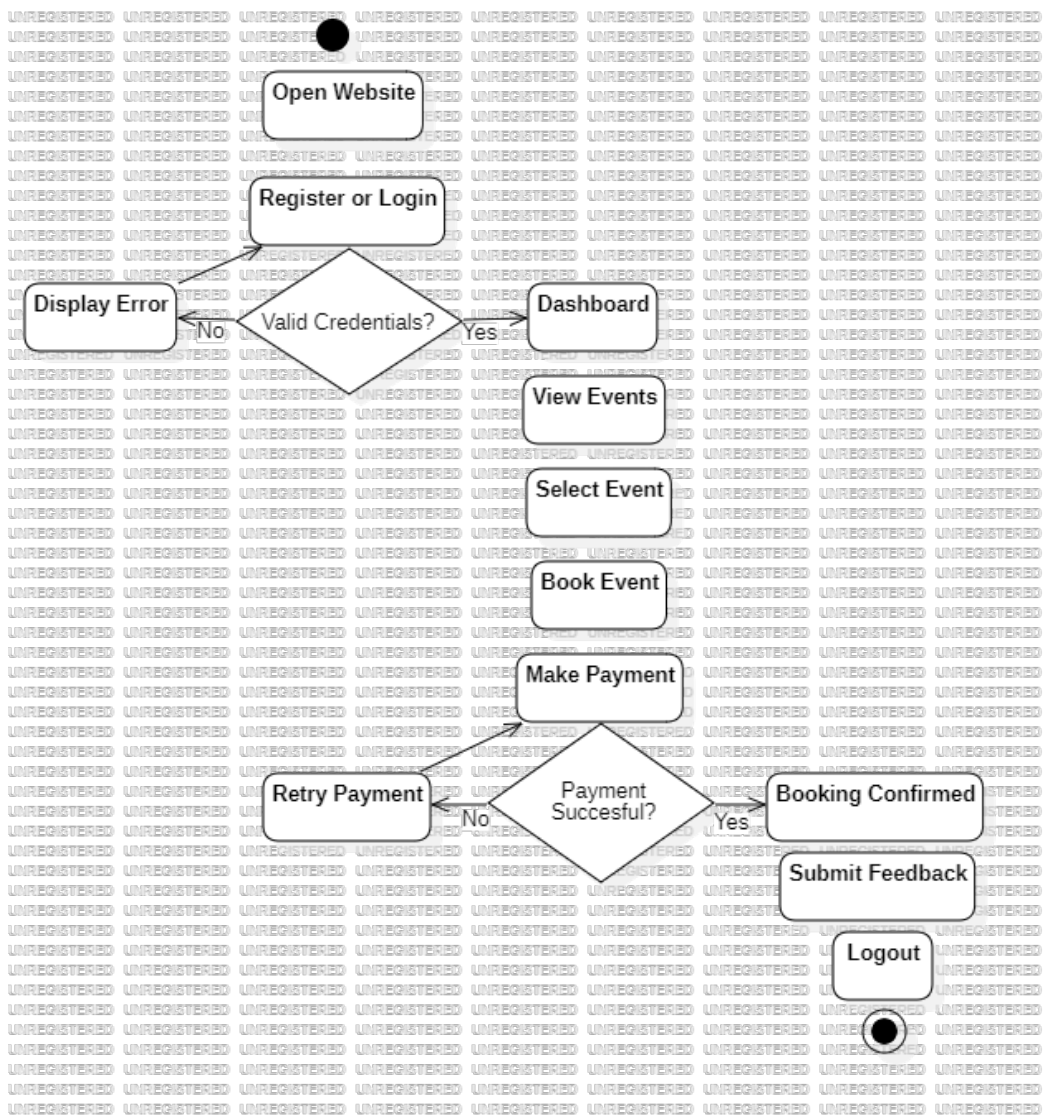


Fig. 8: Activity Diagram

3.6.7 Sequence Diagram

The sequence diagram shows the ordered exchange of messages between the participants — the User, the Frontend, the Backend, the Database and the Payment Gateway — across the login, event browsing and ticket booking flows. It makes explicit that the frontend never touches the database directly: every read and write is mediated by the backend, which is where validation and authorisation are enforced.

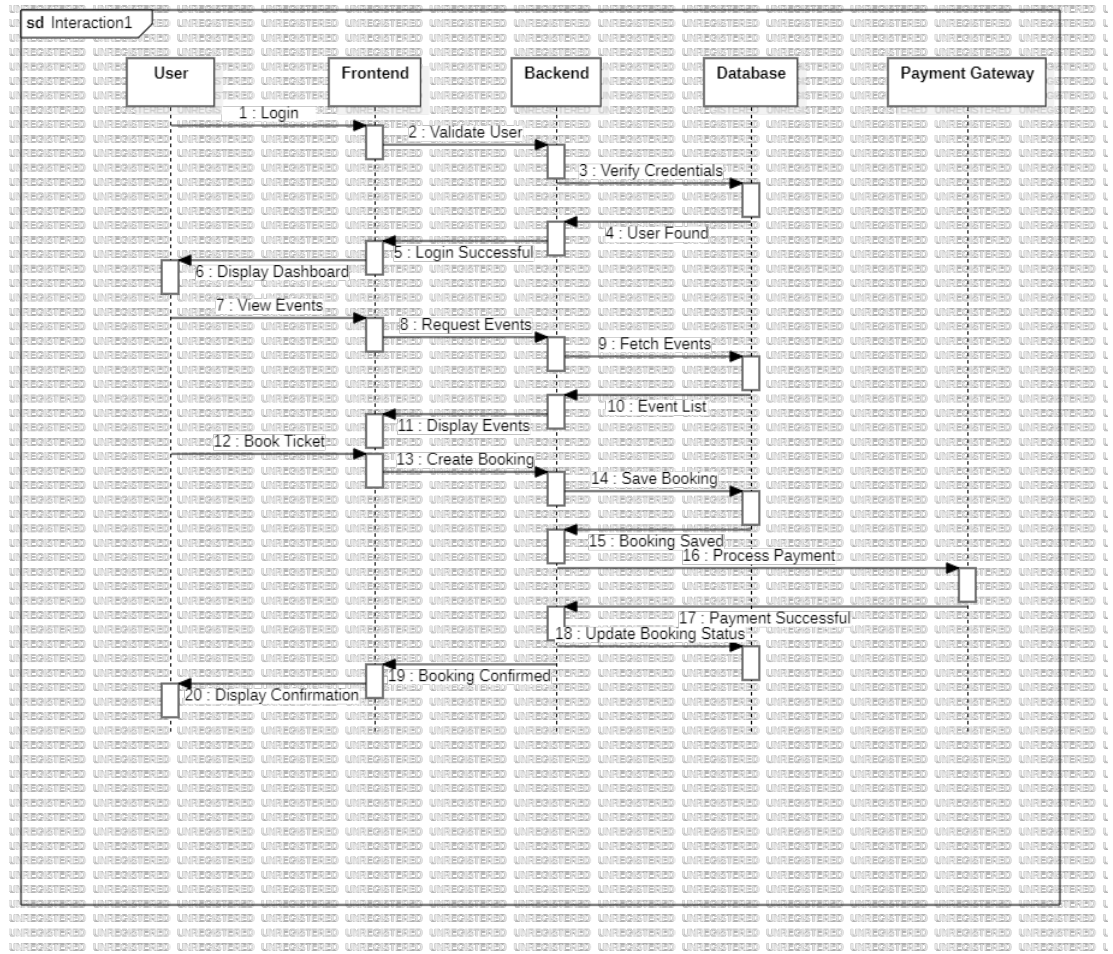
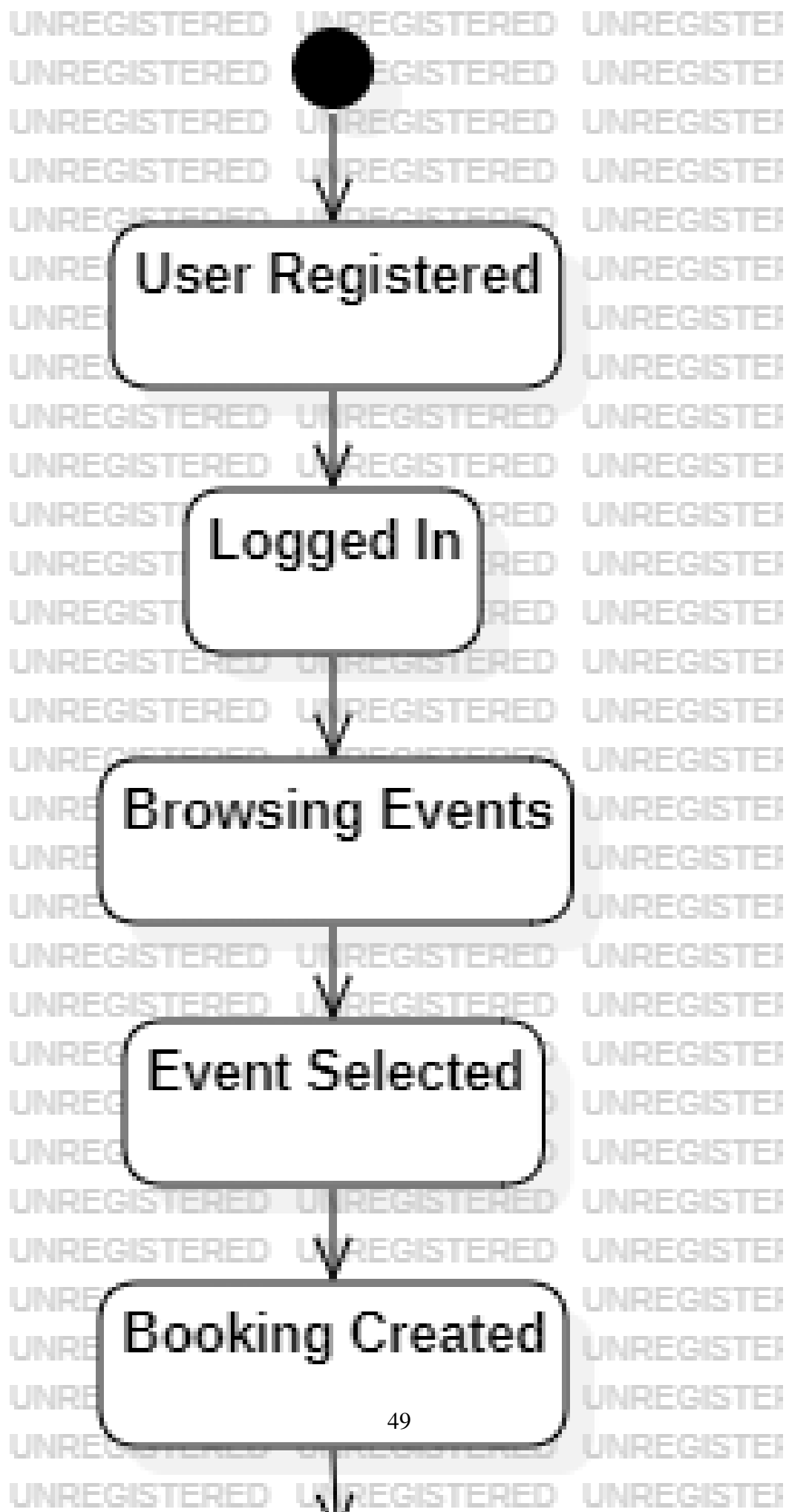


Fig. 9: Sequence Diagram

3.6.8 State Diagram

The state diagram models the lifecycle a user passes through, from registration and login through browsing, selection, booking creation, payment and confirmation to sign-out. The booking states in particular — created, payment pending, payment successful, confirmed — correspond directly to the enumerated status values enforced on the Booking schema in Section 4.2.1.



3.6.9 Package Diagram

The package diagram groups the system's classes into logical packages — Authentication, Event Management, Booking Management, Payment Management, Feedback Management and Admin Management — and shows the dependencies between them. The direction of those dependencies is the point of interest: Booking Management depends on Event Management, and Payment Management on Booking Management, but not the reverse, which is what allows each layer to be modified without disturbing the one below it.

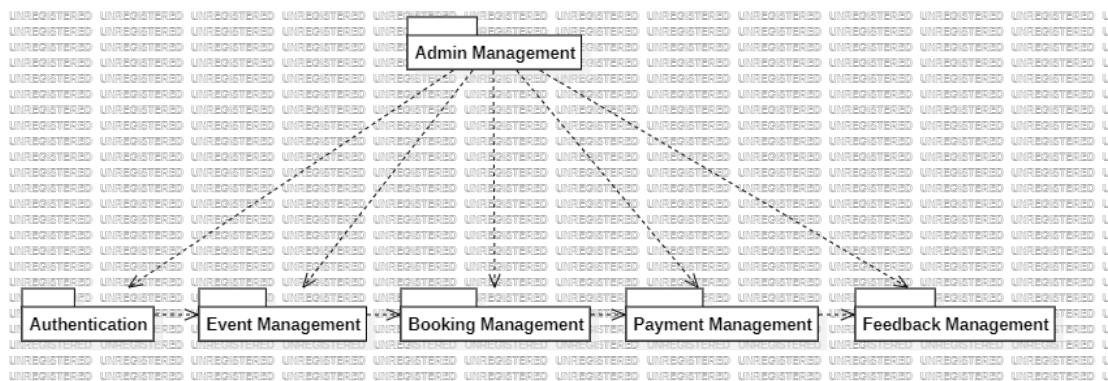


Fig. 11: Package Diagram

3.6.10 Component Diagram

The component diagram shows the deployable software components and the interfaces through which they communicate: the React interface components and their supporting context and service layers on the client, and the Express router, middleware, controllers and Mongoose models on the server, terminating at the MongoDB Atlas data store.

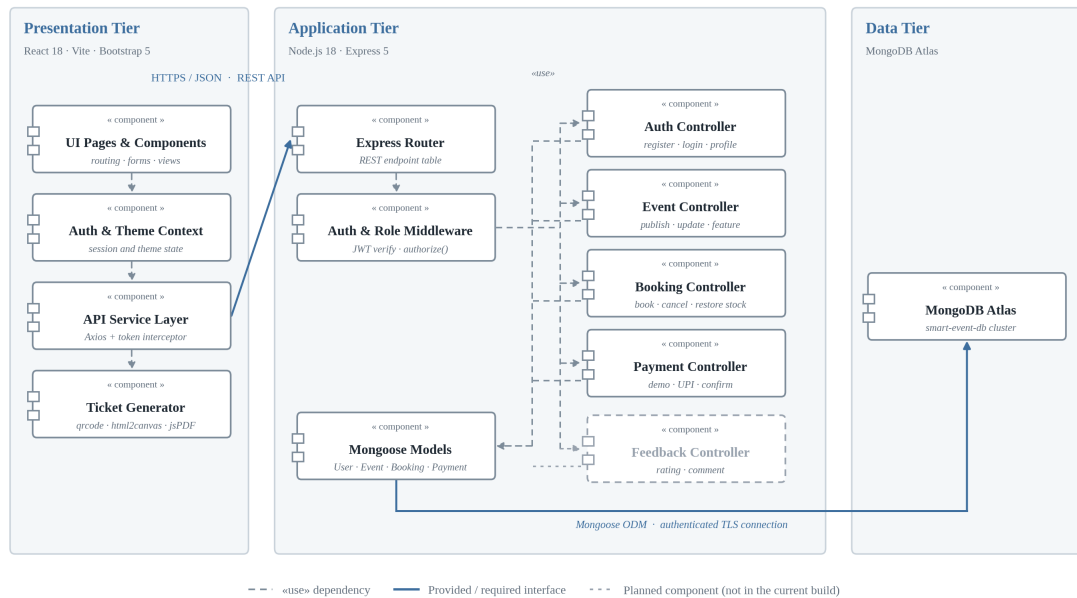


Fig. 12: Component Diagram

3.6.11 Deployment Diagram

The deployment diagram maps the software components onto the physical and virtual nodes that host them: the client device running the browser, the node serving the React frontend, the Node.js and Express application server, and the MongoDB Atlas database cluster, with the communication paths between them.

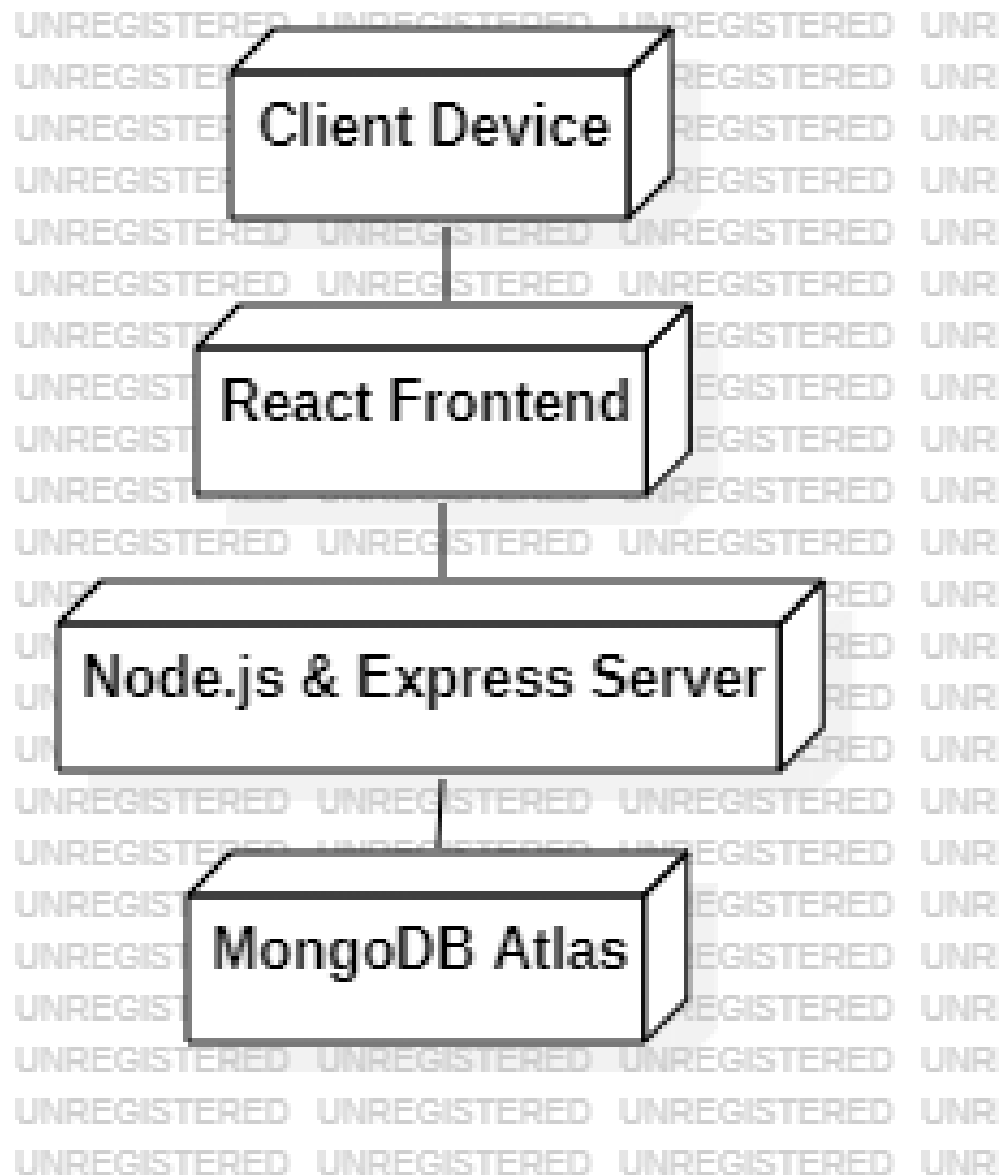


Fig. 13: Deployment Diagram

3.6.12 Data Flow Diagram

The data flow diagram traces how information moves through the system rather than how the system is structured. Two external entities — User and Admin — exchange data with five processes: User Authentication, Event Management, Booking Management, Payment Management and Feedback Management. Each process reads from and writes to its own data store, and the labelled flows between them show exactly what data crosses each boundary. Reading it alongside the sequence diagram in Section 3.6.7 is instructive: the sequence diagram shows the order in which messages are exchanged, while this diagram shows what

information those messages carry and where it comes to rest.

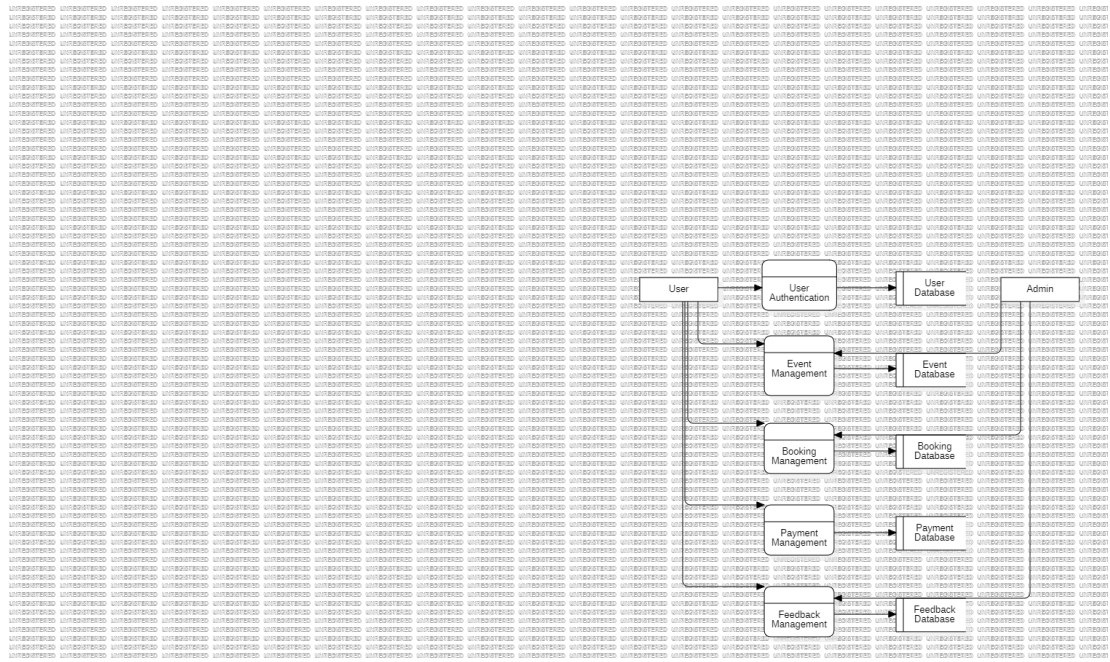


Fig. 14: Data Flow Diagram

CHAPTER 4

SYSTEM DESIGN

4.1 Basic Modules

The system is decomposed into six functional modules. Each owns a distinct area of responsibility, exposes a defined set of operations, and communicates with the others only through those operations. The decomposition follows the package structure presented in Section 3.6.9 and is mirrored directly in the codebase, where each module corresponds to a controller, a set of routes and — where it owns persistent data — a Mongoose model.

4.1.1 Module 1: User Management and Authentication

Purpose. To establish and verify identity, and to enforce the distinction between an ordinary user and an administrator.

Operations. Account registration; user sign-in; administrator sign-in; retrieval of the currently authenticated account; profile image update; sign-out; and administrative retrieval of the registered user list.

Processing. Registration validates the submitted name, email and password, rejects an email that already exists, and stores the account with the password hashed by bcrypt. Sign-in retrieves the account by email, compares the supplied password against the stored hash using bcrypt's constant-time comparison, and on success signs a JSON Web Token carrying the account identifier and role. Every subsequent protected request presents that token, which middleware verifies before the request reaches a controller.

Data. The `users` collection.

Validation. Name of at least two characters; a syntactically valid email address, stored lowercase and unique across the collection; a password of at least six characters matching its confirmation; role constrained to the values `user` or `admin`.

4.1.2 Module 2: Event Management

Purpose. To allow events to be published, maintained and discovered.

Operations. Create event; update event; delete event; list published events; list featured events; retrieve an event by slug; list the events belonging to the signed-in account; list all events for administrators; set and withdraw featured status.

Processing. Creation validates the submitted details, derives a URL-friendly slug from the title, guarantees the slug's uniqueness, initialises the available ticket count to the total, and stores the event with a reference to the account that created it. Update and delete first verify that the caller either owns the event or holds the administrator role. The public listing returns only events whose status is published.

Data. The `events` collection.

Validation. Title present and at most 120 characters; description present and at most 1200 characters; category, venue, city and organiser name present; start and end dates present and well-formed; price zero or greater; total tickets at least one; status drawn from the permitted set.

4.1.3 Module 3: Booking Management

Purpose. To reserve tickets against an event and to keep ticket inventory consistent with the set of active bookings.

Operations. Create booking; list the signed-in user's bookings; retrieve a single booking; cancel a booking; and, for administrators, list all bookings and create, cancel or transfer a booking on a user's behalf.

Processing. A booking request is checked against the event's current available ticket count before anything is written. If sufficient tickets remain, the booking is created with a generated reference code and a snapshot of the event's title, slug, start date, venue and city; the event's available count is decremented by the booked quantity; and the event's status becomes `sold_out` if the count reaches zero. Cancellation reverses this: the booking's status becomes `cancelled`, its payment is marked refunded, the quantity is returned to the event, and an event previously marked sold out returns to published.

Data. The `bookings` collection; reads from `events` and `users`; writes to `payments` through Module 4.

Validation. The caller must be authenticated; the event must exist and be bookable; quantity must be a positive integer not exceeding availability; a priced event must be settled through the payment module rather than booked directly; a booking already cancelled cannot be cancelled again.

4.1.4 Module 4: Payment Management

Purpose. To settle the amount owed on a booking and to maintain an auditable record of every payment attempt.

Operations. Complete a demo-gateway payment; submit a manual UPI transfer reference; confirm a reviewed UPI payment (administrator); list the payment ledger (administrator).

Processing. Three channels are supported. A zero-priced event produces a payment record with status `not_required` and the booking is confirmed immediately. The demo gateway records a payment with status `paid` and confirms the booking in the same operation. The manual UPI channel records a payment with status `pending` against the transfer reference the customer supplies, leaving the booking in `pending_payment` until an administrator reviews the transfer and confirms it, at which point the payment becomes `paid`, the confirming administrator and timestamp are recorded, and the booking is confirmed.

Data. The `payments` collection; updates bookings.

Validation. The referenced booking must exist and belong to the caller; the amount must equal the booking total; the provider and method must be drawn from their permitted sets; a payment already confirmed cannot be confirmed twice.

4.1.5 Module 5: Ticketing

Purpose. To issue the artefact the attendee actually carries.

Operations. Render a ticket for a confirmed booking; encode its reference as a QR code; export the ticket as a PDF.

Processing. The ticket component renders the booking reference, event title, date, venue and quantity, alongside a QR code encoding the booking's identifying data. When the holder requests a copy, the rendered element is rasterised to a canvas and placed into a PDF sized to fit it. All three supporting libraries are loaded on demand, so the code is fetched only by users who actually request a ticket.

Data. Reads from bookings; writes nothing.

Validation. Only a booking whose status is confirmed yields a ticket.

4.1.6 Module 6: Administration

Purpose. To give administrators oversight of the platform.

Operations. List all events, bookings, payments and registered users; edit or delete any event; control featured placement; confirm reviewed UPI payments; create, cancel or transfer bookings on a user's behalf; and query system health.

Processing. Every administrative route passes through the token verification middleware and then through role authorisation, so administrative capability is enforced at the API layer rather than assumed from the interface. The administrator account itself is provisioned from environment configuration when the server first connects to the database, which avoids shipping any credential in the codebase. The administrative sign-in page is deliberately unlinked from the public navigation and is reached through a hidden trigger.

Data. Reads across all collections; writes to events, bookings and payments.

Validation. The caller's role must be admin on every operation in this module.

4.1.7 Module 7: Feedback Management (Planned)

Purpose. To let an attendee rate and comment on an event after attending, and to let administrators review what has been submitted.

Operations. Submit feedback; edit one's own feedback; list feedback for an event; list all feedback (administrator).

Data. A feedback collection holding a reference to the user, a reference to the event, an integer rating, a comment and a timestamp.

Validation. One feedback record per user per event; the user must hold a confirmed booking for the event; the rating must fall within the permitted range.

This module is specified in the conceptual models of Section 3.6 and is carried through the design for completeness. It is scheduled for the next development iteration and does not form part of the implementation described in Chapter 5.

4.2 Data Design

4.2.1 Schema Design

The system stores its data in MongoDB, a document-oriented database, accessed through the Mongoose object data modelling library. MongoDB itself does not enforce a structure on the documents within a collection; Mongoose adds a schema layer above it that declares the permitted fields, their types, whether they are required and what values they may hold. The application therefore has the flexibility of a document store while retaining the field-level guarantees usually associated with a relational system.

The database is organised into four collections — users, events, bookings and payments — each backed by a schema defined under backend/models/. Every schema is declared with the timestamps option, so each document automatically carries createdAt and updatedAt fields maintained by Mongoose rather than by application code.

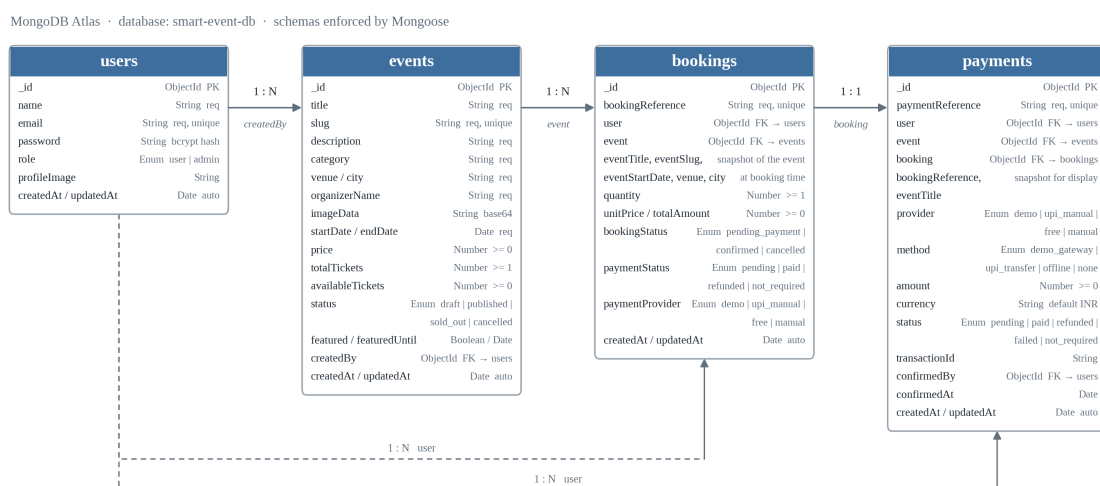


Fig. 15: Database Schema Design

User Schema

The User schema stores authentication credentials and the role that separates an ordinary user from an administrator.

Field	Type	Constraints	Description
_id	ObjectId	primary key, generated	Unique document identifier

Field	Type	Constraints	Description
name	String	required, 2–50 characters	Display name of the account holder
email	String	required, unique, lowercase, pattern-validated	Login identifier
password	String	required, minimum 6 characters	bcrypt hash; never stored in plain text
role	String (enum)	user admin, default user	Determines access to administrative routes
profileImage	String	optional, default empty	Stored image reference for the avatar
createdAt, updatedAt	Date	auto-managed	Creation and last-modified timestamps

A `pre('save')` hook hashes the password with bcrypt using ten salt rounds whenever it is created or modified, so a plain-text password never reaches the database. A `comparePassword()` instance method performs the corresponding comparison at login, and a `toSafeObject()` method strips the hash before any user document is returned to a client.

Event Schema

Field	Type	Constraints	Description
title	String	required, max 120 characters	Event name
slug	String	required, unique	URL-friendly identifier derived from the title
description	String	required, max 1200 characters	Full event description
category	String	required, max 60 characters	Event category
venue, city	String	required	Physical location
organizerName	String	required, max 80 characters	Organising entity
imageData	String	optional	Base64-encoded banner image
startDate, endDate	Date	required	Start and end date-time
price	Number	required, minimum 0	Ticket price
totalTickets	Number	required, minimum 1	Capacity at listing time

Field	Type	Constraints	Description
availableTickets	Number	required, minimum 0	Remaining unsold tickets
status	String (enum)	draft published sold_out cancelled	Current lifecycle state
featured, featuredUntil	Boolean, Date	optional	Homepage featured placement and its expiry
createdBy	ObjectId → User	optional	Account that published the event

Booking Schema

Field	Type	Constraints	Description
bookingReference	String	required, unique	Human-readable code, e.g. BKG-20260806-D5GIU2
user	ObjectId → User	required	Account that made the booking
event	ObjectId → Event	required	Event being booked
eventTitle, eventSlug, eventStartDate, venue, city	String, Date	required, denormalised	Snapshot of the event at booking time
quantity	Number	required, minimum 1	Number of tickets
unitPrice, totalAmount	Number	required, minimum 0	Price per ticket and computed total
bookingStatus	String (enum)	pending_payment confirmed cancelled	Lifecycle state of the booking
paymentStatus	String (enum)	pending paid refunded not_required	Payment state
paymentProvider	String (enum)	demo upi_manual free manual	Channel used
paymentId, paymentOrderId	String	optional	External payment identifiers

Payment Schema

Field	Type	Constraints	Description
paymentReference	String	required, unique	Human-readable code, e.g. PAY-20260825-YZ0FKL
user, event, booking	ObjectId	required	References tying the payment to its context
bookingReference, eventTitle	String	required, denormalised	Snapshot fields for display without a join
provider	String (enum)	demo upi_manual free manual	Payment provider
method	String (enum)	demo_gateway upi_transfer offline none	Payment method
amount	Number	required, minimum 0	Transaction amount
currency	String	default INR, upper-case	ISO currency code
status	String (enum)	pending paid refunded failed not_required	Current transaction state
transactionId	String	optional	Provider-side reference
confirmedBy, confirmedAt	ObjectId → User, Date	optional	Administrator who confirmed the payment, and when
refundedAt	Date	optional	Timestamp of refund, if applicable

The Payment schema additionally declares compound indexes on {booking, createdAt}, {user, createdAt} and {status}, so that the three queries the application actually issues — a booking’s payment record, a user’s payment history, and filtering by status in the administrative ledger — remain efficient as the collection grows.

Design Rationale

Denormalisation in preference to joins. MongoDB does not perform relational joins, and Mongoose’s `populate()` — its nearest equivalent — issues a separate query for each reference resolved. To avoid that cost on frequently-read collections, Booking and Payment documents duplicate a small set of Event fields at the moment they are created. This buys

more than performance. Because the copy is taken at purchase time, a historical booking continues to show what the customer actually bought even after the event listing is edited. Without it, changing an event's venue would silently rewrite the record of every past booking against that event.

A single collection for both roles. Ordinary users and administrators are stored in the same `users` collection and distinguished only by the `role` field, rather than being separated into two collections. Access control is therefore enforced consistently at one place — the route middleware — instead of being implied by which collection a record happens to live in.

Enumerated status fields. Every lifecycle-sensitive field — `Event.status`, `Booking.bookingStatus` and `paymentStatus`, and `Payment.status` — is constrained by a Mongoose enumeration validator. This pushes state-machine validation into the schema itself, so an invalid value is rejected at the point of writing rather than discovered later by whatever code reads it.

Referential integrity without foreign keys. MongoDB enforces no foreign-key constraints. Referential correctness is maintained instead at the application layer: controllers create a `Booking` or a `Payment` only after confirming that the referenced `User` and `Event` exist, and every reference field is marked `required` so that a missing reference cannot be persisted through the normal application flow.

4.2.2 Data Integrity and Constraints

Data integrity in this system is enforced at three levels, each catching a different class of error.

Schema-level constraints are declared on the Mongoose models and applied on every write. Required fields cannot be omitted. The `email` field carries a uniqueness constraint, which prevents duplicate accounts even if two registration requests arrive simultaneously and both pass the application's own duplicate check. String fields carry length bounds, numeric fields carry minimum values — `price` at least zero, `totalTickets` at least one, `availableTickets` at least zero — and every status field is restricted to an enumerated set. The `email` field is additionally validated against a pattern and normalised to lowercase before storage, so that address casing cannot produce two accounts that are the same address in practice.

Application-level constraints are enforced in the controllers, and cover the rules a schema cannot express. Registration rejects an email already present in the collection with a specific

error rather than allowing the database constraint to surface as an opaque failure. Booking creation rejects a quantity exceeding the event’s available tickets, and rejects a direct booking against a priced event that has not been paid for. Cancellation rejects a booking already cancelled. Payment confirmation rejects a payment already confirmed. Update and delete operations on an event verify that the caller either created that event or holds the administrator role.

Invariant maintenance concerns the one relationship the system must never allow to drift: the correspondence between an event’s available ticket count and the set of active bookings against it. Every operation that creates a booking decrements the count in the same request that writes the booking, and every cancellation restores it and reverses the sold-out status if it was set. Because both halves of each operation are performed by the same controller before it responds, the count and the bookings cannot diverge through the normal application flow.

Credential integrity is maintained by construction. The password field is hashed by a pre-save hook that runs on every create and every modification, so there is no code path through which a plain-text password can be written — not even one that bypasses the controller. Verification uses bcrypt’s own comparison rather than string equality, and the `toSafeObject()` projection ensures the hash is never included in a response.

4.3 User Interface Design

4.3.1 Design Principles

Four principles governed the interface design.

Progressive disclosure of capability. A visitor who has not signed in sees the landing page, the events listing and individual event pages. Booking, ticket download, event publishing and administrative consoles appear only once the corresponding role is established. Capability is never merely hidden — the same distinction is enforced at the API — but the interface avoids presenting controls that would fail.

Consistency of structure. Every authenticated page shares a common layout: a persistent navigation bar carrying the brand, the primary links, the theme toggle and the account menu; a content region; and a footer. Two layout components, `MainLayout` and `AuthLayout`, implement this, so the structure cannot drift between pages.

Responsiveness by default. The interface is built on Bootstrap’s twelve-column grid with mobile-first breakpoints. Event listings collapse from a multi-column grid to a single column on narrow viewports; the navigation collapses to a toggle; tables in the administrative consoles scroll horizontally within their container rather than forcing the page to scroll.

Immediate, specific feedback. Every form validates on the client before submission and displays server-side validation messages inline. Actions that take time show a loading state rather than appearing unresponsive, and every destructive action — cancelling a booking, deleting an event — requires confirmation.

4.3.2 Input Design

Form	Fields	Validation applied
Registration	Name, email, password, confirm password	All required; name ≥ 2 characters; email pattern-checked and checked for duplication; password ≥ 6 characters and must match its confirmation
User sign-in	Email, password	Both required; email pattern-checked; credentials verified against the stored hash

Form	Fields	Validation applied
Administrator sign-in	Email, password	As above, with an additional role check
Create / edit event	Title, description, category, organiser, venue, city, banner image, start and end date-time, price, total tickets	All required except the banner; length bounds on text fields; end date after start date; price ≥ 0 ; tickets ≥ 1
Booking	Ticket quantity	Positive integer; must not exceed available tickets
Demo gateway payment	Confirmation of the displayed amount	Amount must match the booking total
Manual UPI payment	Transfer reference	Required; recorded against the booking for administrative review
Profile image	Image file	Encoded and stored against the account

4.3.3 Output Design

The system's outputs are the event listing and detail pages; the user dashboard; the booking history; the QR-coded ticket and its PDF export; payment confirmations; the four administrative consoles for events, bookings, payments and users; and the validation and error messages produced when an operation cannot be completed. Every output is rendered through the same Bootstrap-based component set, so presentation is consistent across desktop, tablet and mobile.

4.3.4 Screen Design

[Section pending — interface screenshots]

This section presents the implemented screens: the landing page, the events listing, an event detail page, the sign-in and registration forms, the administrator sign-in page, the user dashboard, the booking and payment screens for both channels, the booking history, the QR-coded ticket, the event creation form, and the responsive and dark-theme presentations. Screenshots are to be inserted here.

4.4 Security Issues

Security in this system is addressed at six layers. Each is described below together with the specific threat it is intended to counter.

4.4.1 Password Storage

Passwords are never stored or transmitted in recoverable form. A `pre('save')` hook on the User schema hashes the password with `bcrypt` using ten salt rounds before the document is written, and because the hook is attached to the schema rather than invoked by a controller, there is no code path that can write a plain-text password. `bcrypt` incorporates a distinct random salt into every hash, so two accounts sharing a password produce different stored values and precomputed lookup tables are useless. Verification uses `bcrypt`'s constant-time comparison rather than string equality, which prevents an attacker from inferring a password character by character from response timing.

This counters the compromise of the database itself: an attacker holding the users collection obtains hashes whose reversal costs the `bcrypt` work factor per candidate password tested.

4.4.2 Authentication

Authentication is stateless and token-based. A successful sign-in produces a JSON Web Token signed with a secret held in environment configuration and carrying the account identifier and role. The client presents that token in the `Authorization` header of every subsequent request; the `protect` middleware verifies the signature, loads the corresponding account with the password field explicitly excluded, and attaches it to the request. A request with no token, a malformed token, an expired token or a token whose subject no longer exists is rejected with 401 before any controller runs.

4.4.3 Authorisation

Authentication establishes who is calling; authorisation determines what they may do. The `authorize()` middleware is a factory returning a guard for a given set of roles, and it is composed into the route definition after `protect`. Administrative routes are therefore declared as the path followed by `protect`, then `authorize('admin')`, then the controller.

Two aspects of this deserve emphasis. First, authorisation is enforced at the API and not in the interface: hiding an administrative link does not protect the endpoint behind it, and the endpoint is protected independently. Second, ownership checks supplement role checks where roles alone are insufficient — updating or deleting an event verifies that the caller created it or is an administrator, so one user cannot modify another’s event merely by holding a valid token.

4.4.4 Configuration and Secret Management

The database connection string, the token signing secret and the administrator bootstrap credentials are supplied through environment variables loaded at start-up, and the environment file is excluded from version control. No credential appears in the codebase. The administrator account is created from that configuration when the server first connects to the database, which avoids the alternative of shipping a seeded account with a known password.

4.4.5 Transport and Origin Control

Both deployed tiers are served over HTTPS, so credentials and tokens are encrypted in transit. Cross-Origin Resource Sharing is configured to accept browser requests only from the configured frontend origin, which prevents a page hosted elsewhere from issuing authenticated requests against the API using a victim’s stored token.

4.4.6 Input Handling and Error Disclosure

Every endpoint validates its input before acting on it, and Mongoose’s schema validators provide a second layer beneath that. Because MongoDB is queried through Mongoose’s typed API rather than by string concatenation, the injection route familiar from SQL systems does not arise in the same form. A centralised error middleware formats every failure into a consistent JSON response, so stack traces and internal paths are never returned to a client, and a not-found middleware handles unmatched routes uniformly.

4.4.7 Known Limitations

Two limitations are worth stating explicitly. Because authentication is stateless, a token cannot be revoked before it expires without reintroducing server-side state; a compromised

token remains valid for the remainder of its lifetime. And rate limiting is not currently applied to the authentication endpoints, so the system does not itself slow an automated credential-guessing attempt — although bcrypt’s cost factor makes such an attempt expensive regardless. Both are noted in Section 7.2 and addressed in Section 7.3.

4.5 Test Cases Design

Test cases were designed to cover each module's normal path, its boundary conditions and its failure modes. The table below records the case identifier, the module under test, the scenario, the input supplied, the expected result and the outcome observed. The results obtained when these cases were executed are reported in Section 6.1.

ID	Module	Scenario	Input	Expected Result	Status
TC-01	User Management	Register with valid details	Name, unused email, matching passwords	Account created and user signed in	Pass
TC-02	User Management	Register with a duplicate email	Email already registered	Rejected with a duplicate-account message	Pass
TC-03	User Management	Register with a malformed email	user@@mail	Rejected with a format message	Pass
TC-04	User Management	Register with mismatched passwords	Password and confirmation differ	Rejected with a mismatch message	Pass
TC-05	User Management	Register with a short password	4-character password	Rejected with a minimum-length message	Pass
TC-06	User Management	Sign in with valid credentials	Registered email and password	Token issued and dashboard displayed	Pass
TC-07	User Management	Sign in with a wrong password	Valid email, wrong password	Rejected with an invalid-credentials message	Pass
TC-08	User Management	Administrator sign-in	Bootstrap admin credentials	Administrative dashboard displayed	Pass
TC-09	User Management	Access a protected route without a token	No Authorization header	401 Unauthorised	Pass
TC-10	User Management	Access an admin route as an ordinary user	Valid user token	403 Forbidden	Pass
TC-11	Event Management	Create an event with valid details	Complete event form	Event created with a unique slug	Pass

ID	Module	Scenario	Input	Expected Result	Status
TC-12	Event Management	Create an event with zero tickets	totalTickets = 0	Rejected by the minimum-value validator	Pass
TC-13	Event Management	Create an event with a negative price	price = -100	Rejected by the minimum-value validator	Pass
TC-14	Event Management	Create two events with the same title	Duplicate title	Both created with distinct slugs	Pass
TC-15	Event Management	Update an event owned by another user	Valid token, another user's event	403 Forbidden	Pass
TC-16	Event Management	Delete an own event	Owner's token	Event removed from the listing	Pass
TC-17	Event Management	Retrieve an event by slug	Valid published slug	Event detail returned	Pass
TC-18	Event Management	Retrieve an unpublished event publicly	Draft event slug	Not returned in the public listing	Pass
TC-19	Booking Management	Book within availability	Quantity 2, 10 available	Booking confirmed; availability becomes 8	Pass
TC-20	Booking Management	Book exactly the remaining tickets	Quantity 3, 3 available	Booking confirmed; event becomes sold out	Pass
TC-21	Booking Management	Book more than available	Quantity 5, 3 available	Rejected with an availability message	Pass
TC-22	Booking Management	Book zero tickets	Quantity 0	Rejected by quantity validation	Pass
TC-23	Booking Management	Book without signing in	No token	401 Unauthorised	Pass
TC-24	Booking Management	View own booking history	Valid user token	Only that user's bookings returned	Pass
TC-25	Booking Management	Cancel a booking	Confirmed booking	Cancelled; tickets restored; sold-out event republished	Pass
TC-26	Booking Management	Cancel an already-cancelled booking	Cancelled booking	Rejected with an already-cancelled message	Pass

ID	Module	Scenario	Input	Expected Result	Status
TC-27	Payment Management	Book a free event	price = 0	Confirmed with payment status not_required	Pass
TC-28	Payment Management	Pay through the demo gateway	Valid paid booking	Payment recorded as paid; booking confirmed	Pass
TC-29	Payment Management	Submit a manual UPI reference	Transfer reference	Payment recorded as pending; booking held	Pass
TC-30	Payment Management	Confirm a reviewed UPI payment	Administrator confirmation	Payment paid; booking confirmed; confirmer recorded	Pass
TC-31	Payment Management	Confirm the same payment twice	Already-confirmed payment	Rejected	Pass
TC-32	Payment Management	Verify the ledger after cancellation	Cancelled paid booking	Payment marked refunded	Pass
TC-33	Ticketing	Generate a ticket for a confirmed booking	Confirmed booking	Ticket rendered with a scannable QR code	Pass
TC-34	Ticketing	Export a ticket as PDF	Ticket view	PDF downloaded matching the on-screen ticket	Pass
TC-35	Administration	List all events	Administrator token	Every event returned regardless of status	Pass
TC-36	Administration	List all bookings	Administrator token	Every booking on the platform returned	Pass
TC-37	Administration	List the payment ledger	Administrator token	Complete ledger returned	Pass
TC-38	Administration	List registered users	Administrator token	Every account returned without password hashes	Pass
TC-39	Administration	Transfer a booking	Source and target accounts	Booking reassigned to the target account	Pass
TC-40	System	Query the health endpoint	GET /api/health	API and database status reported	Pass

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approach

5.1.1 Strategy

Implementation followed an incremental, vertically-sliced approach. Rather than building the entire backend and then the entire frontend, each feature was carried from the database schema through the API to the interface before the next feature was started. Authentication was completed end to end first; then event publishing and discovery; then booking; then payments; then the administrative consoles; then ticketing.

The reason for working this way is that it keeps the system runnable at every point. A horizontal approach — all models, then all controllers, then all pages — leaves nothing demonstrable until very late, and defers the discovery of interface-level problems until the API is already fixed. Working vertically meant that after the first week there was a working application that could be signed into, and every subsequent slice added capability to something that already ran. It also meant that API design decisions were validated against a real consumer immediately, rather than against an imagined one.

Within each slice the order was consistent: define or extend the Mongoose schema; write the controller and its validation; register the route with its middleware; test the endpoint in Postman across valid input, invalid input, missing token and wrong role; add the service function on the frontend; and finally build the interface that calls it. Testing the endpoint before building the interface is what makes a subsequent failure unambiguous — if the endpoint was verified in isolation, a failure in the browser is an interface problem.

5.1.2 Architecture

The implementation is a three-tier client–server application.

The **presentation tier** is a React single-page application. It is organised into pages/ for route-level views, components/ for reusable interface elements, layouts/ for the two

shared page structures, `context/` for global state, `hooks/` for the consumers of that state, `services/` for the API client and its per-domain call wrappers, and `utils/` for helpers. This separation means an interface change touches `pages/` or `components/`, and an API change touches `services/`, without the two interfering.

The **application tier** is an Express server organised into `routes/` declaring the endpoint table, `controllers/` holding the request handling and business logic, `models/` holding the Mongoose schemas, `middleware/` holding the cross-cutting concerns, `config/` holding the database connection and the administrator bootstrap, and `utils/` holding token generation and the payment ledger helpers.

The **data tier** is a MongoDB Atlas cluster reached only by the application tier.

The tiers exchange JSON over HTTPS. Because the API holds no session state, identity travels in a signed token attached to each request.

5.1.3 Request Lifecycle

A single authenticated request illustrates how the pieces fit together. A user submits the booking form. The page calls a function in `services/bookings.js`, which issues an Axios request through the shared client; the client's request interceptor attaches the stored token as an `Authorization` header. The request reaches Express, passes through CORS negotiation, JSON body parsing and request logging, and matches a route in `bookingRoutes.js`. That route names `protect`, which verifies the token signature and loads the account, and then `authorize('user')`, which checks the role. Only then does the controller run: it validates the quantity, loads the event, checks availability, creates the booking, decrements the event's ticket count, records the payment, and responds. Any error thrown along the way — including from a rejected promise — reaches the centralised error middleware, which formats it into a consistent JSON response. Back in the browser, the page updates from the response.

5.2 Coding Details and Code Efficiency

This section presents the code behind the mechanisms described above. Listings are given for the parts of the system where the implementation is not obvious from the design.

5.2.1 Password Hashing

Hashing is attached to the schema rather than performed in a controller, so that no code path can write a plain-text password.

```
1 userSchema.pre('save', async function savePassword(next) {
2   if (!this.isModified('password')) {
3     next();
4     return;
5   }
6
7   const salt = await bcrypt.genSalt(10);
8   this.password = await bcrypt.hash(this.password, salt);
9   next();
10 });
11
12 userSchema.methods.comparePassword = async function comparePassword(
13   candidatePassword) {
14   return bcrypt.compare(candidatePassword, this.password);
15 };
16
17 userSchema.methods.toSafeObject = function toSafeObject() {
18   return {
19     _id: this._id,
20     name: this.name,
21     email: this.email,
22     role: this.role,
23     profileImage: this.profileImage,
24     createdAt: this.createdAt,
25     updatedAt: this.updatedAt
26   };
27 };
28 }
```

Listing 5.1: Password hashing and verification — backend/models/User.js

The guard on `isModified` matters: without it, every save of an existing user would re-hash

the already-hashed value, and the stored password would become unusable after any unrelated profile update. `toSafeObject()` is the projection applied before any user document is returned, so the hash cannot leak through a response by oversight.

5.2.2 Token Issue and Verification

```
1 import jwt from 'jsonwebtoken';
2
3 const generateToken = (userId, role) =>
4   jwt.sign({ userId, role }, process.env.JWT_SECRET, {
5     expiresIn: process.env.JWT_EXPIRES_IN || '7d'
6   });
7
8 export default generateToken;
```

Listing 5.2: Token generation — `backend/utils/generateToken.js`

```
1 export const protect = async (req, res, next) => {
2   const authHeader = req.headers.authorization;
3
4   if (!authHeader || !authHeader.startsWith('Bearer ')) {
5     res.status(401);
6     next(new Error('Access denied. Token is missing or invalid.'));
7     return;
8   }
9
10  try {
11    const token = authHeader.split(' ')[1];
12    const decoded = jwt.verify(token, process.env.JWT_SECRET);
13    const user = await User.findById(decoded.userId).select('-password');
14
15    if (!user) {
16      res.status(401);
17      next(new Error('User linked to this token no longer exists.'));
18      return;
19    }
20
21    req.user = user;
22    next();
23  } catch (error) {
24    res.status(401);
```

```

25   next(new Error('Unauthorized access. Please log in again.'));
26   }
27 };
28
29 export const authorize = (...roles) => (req, res, next) => {
30   if (!req.user || !roles.includes(req.user.role)) {
31     res.status(403);
32     next(new Error('You are not authorized to access this resource.'));
33     return;
34   }
35
36   next();
37 };

```

Listing 5.3: Authentication and authorisation middleware —
backend/middleware/authMiddleware.js

authorize is a factory: calling it with a set of roles returns a middleware function closed over those roles. This is what allows a route to be declared declaratively, with its protection visible in the route table rather than buried in a controller:

```

1 router.get('/admin/list', protect, authorize('admin'), getAdminBookings);
2 router.post('/admin/:id/transfer', protect, authorize('admin'),
   transferAdminBooking);
3
4 router.get('/', protect, authorize('user'), getMyBookings);
5 router.post('/', protect, authorize('user'), createBooking);
6 router.post('/:id/cancel', protect, authorize('user'), cancelBooking);

```

Listing 5.4: Route declaration with composed guards —
backend/routes/bookingRoutes.js

Note that protect re-loads the user from the database on every request rather than trusting the role encoded in the token. A token issued before a role changed therefore cannot be used to exercise the old role.

5.2.3 Ticket Inventory Conservation

The invariant that available tickets must always match the set of active bookings is maintained by pairing every decrement with a corresponding restore.

```

1 const restoreInventoryForBooking = async (booking) => {
2   const event = await Event.findById(booking.event);
3
4   if (!event) {
5     return;
6   }
7
8   event.availableTickets += booking.quantity;
9
10  if (event.status === 'sold_out') {
11    event.status = 'published';
12  }
13
14  await event.save();
15 };
16
17 const cancelExistingBooking = async (booking) => {
18   if (booking.bookingStatus === 'cancelled') {
19     return { error: 'This booking has already been cancelled.' };
20   }
21
22   booking.bookingStatus = 'cancelled';
23   booking.paymentStatus = 'refunded';
24   await booking.save();
25   await restoreInventoryForBooking(booking);
26   await markPaymentsRefunded(booking._id);
27
28   return { booking: await getPopulatedBookingById(booking._id) };
29 };

```

Listing 5.5: Cancellation and inventory restoration —
backend/controllers/bookingController.js

Three things happen together here, and the order is deliberate. The booking is marked cancelled first, so that a concurrent request cannot cancel it twice. The tickets are then returned to the event, and the sold-out status is reversed if it was set — without this second step, an event that sold out would remain unbookable even after cancellations freed capacity. Finally the associated payments are marked refunded, so the ledger reflects the cancellation.

5.2.4 Payment Ledger

Every payment attempt, across all three channels, is written through one helper, which is what keeps the ledger uniform.

```
1 const generatePaymentReference = () => {
2   const datePart = new Date().toISOString().slice(0, 10).replace(/-/g, '');
3   const randomPart = Math.random().toString(36).slice(2, 8).toUpperCase();
4   return `PAY-${datePart}-${randomPart}`;
5 };
6
7 const methodByProvider = {
8   demo: 'demo_gateway',
9   upi_manual: 'upi_transfer',
10  manual: 'offline',
11  free: 'none'
12 };
13
14 export const recordPayment = async ({ booking, event, userId, provider, amount,
15                                     status, transactionId, confirmedBy }) =>
16   Payment.create({
17     paymentReference: generatePaymentReference(),
18     user: userId,
19     event: event._id,
20     booking: booking._id,
21     bookingReference: booking.bookingReference,
22     eventTitle: event.title,
23     provider,
24     method: methodByProvider[provider] || 'none',
25     amount,
26     status,
27     transactionId,
28     confirmedBy,
29     confirmedAt: status === 'paid' ? new Date() : undefined
30   });
```

Listing 5.6: Payment recording — backend/utils/paymentLedger.js

The `methodByProvider` lookup replaces what would otherwise be a conditional chain in each of the three payment controllers. Because the mapping is declared once, adding a fourth provider is a single-line change rather than an edit to every call site — and, more

importantly, the three existing channels cannot drift out of agreement about which method corresponds to which provider.

5.2.5 Token Attachment on the Client

```
1 const api = axios.create({
2   baseURL: import.meta.env.VITE_API_BASE_URL || "/api",
3   timeout: 10000,
4 });
5
6 export const setApiAuthToken = (token) => {
7   authToken = token;
8 };
9
10 api.interceptors.request.use((config) => {
11   if (authToken) {
12     config.headers.Authorization = `Bearer ${authToken}`;
13   }
14   return config;
15 });
```

Listing 5.7: Axios instance with a request interceptor — `src/services/api.js`

Without the interceptor, every one of the roughly thirty API call sites in the application would have to remember to attach the token, and any that forgot would fail at runtime in a way that looked like an authorisation bug. Centralising it means the token is attached exactly once in the codebase.

5.2.6 Route Guarding on the Client

```
1 function ProtectedRoute({ allowedRoles, redirectPath = '/login' }) {
2   const { isAuthenticated, isLoading, user } = useAuth();
3   const location = useLocation();
4
5   if (isLoading) {
6     return (/* session-loading placeholder */);
7   }
8
9   if (!isAuthenticated) {
10    return <Navigate to={redirectPath} replace state={{ from: location }} />;
11  }
```

```

11   }
12
13   if (allowedRoles && !allowedRoles.includes(user.role)) {
14     const fallbackPath = user.role === 'admin' ? '/admin/dashboard' : '/
      dashboard';
15     return <Navigate to={fallbackPath} replace />;
16   }
17
18   return <Outlet />;
19 }

```

Listing 5.8: Client-side route guard — `src/components/ProtectedRoute.jsx`

The explicit `isLoading` branch is what prevents a visible flicker: while the stored token is being exchanged for a session, the component renders a placeholder rather than treating the not-yet-loaded state as unauthenticated and redirecting a legitimate user to the sign-in page. Capturing location in the redirect state allows the user to be returned to the page they were trying to reach after signing in.

This guard is a usability measure, not a security measure. The corresponding API routes are protected independently, as shown above.

5.2.7 Centralised Error Handling

```

1  const errorMiddleware = (error, req, res, next) => {
2    const statusCode = res.statusCode && res.statusCode !== 200 ? res.statusCode
      : 500;
3
4    res.status(statusCode).json({
5      success: false,
6      message: error.message || 'Internal server error.',
7      stack: process.env.NODE_ENV === 'production' ? undefined : error.stack
8    });
9  };

```

Listing 5.9: Error middleware — `backend/middleware/errorMiddleware.js`

Every controller signals failure by setting a status and calling `next()` with an error, and this single handler formats the response. The consequence is that the client can rely on one response shape for every failure, and that stack traces are suppressed in production while

remaining available during development.

5.2.8 Code Efficiency

Efficiency was addressed in five specific ways.

Denormalisation to eliminate query fan-out. Booking and Payment documents store a snapshot of the event fields needed for display. Rendering a user’s booking history therefore requires one query against bookings, not one query plus a populate call per booking. On a history of twenty bookings this is the difference between one round trip and twenty-one.

Targeted indexes. The Payment schema declares compound indexes on {booking, createdAt}, {user, createdAt} and {status} — the three access patterns the application actually uses. Indexes were added for observed queries rather than speculatively, since each one costs write throughput and storage.

Field projection. Queries select only the fields they need. The authentication middleware uses `.select('-password')` so the hash is never even loaded into memory, and the booking controller loads only price when all it needs to know is whether an event is free.

On-demand code loading. The QR code, canvas rasterisation and PDF libraries are loaded through `dynamic import()` at the moment a ticket is requested rather than being bundled into the initial payload. These three libraries together are among the largest dependencies in the project, and the majority of page views never need them.

Single-pass operations. Booking creation performs its availability check, booking write, inventory decrement and payment record within one request rather than across several round trips, so the window in which the count and the bookings could disagree is as narrow as the application can make it.

5.3 Testing Approach

Testing was carried out continuously rather than as a phase at the end. The strategy proceeded from the smallest units outward: individual functions and endpoints first, then the interactions between modules, then the complete system in its deployed environment. This ordering is deliberate — a defect found in an isolated unit has exactly one possible cause, whereas the same defect surfacing in an end-to-end flow could originate anywhere along it.

5.3.1 Unit Testing

Unit testing examined individual functions and endpoints in isolation, with each API route exercised through Postman independently of any interface.

For the authentication module this meant testing registration against valid input, a missing field, a malformed email address, a password shorter than the minimum, a password that did not match its confirmation, and an email already registered. Sign-in was tested with correct credentials, an incorrect password, an unregistered email, and a missing field. Token verification was tested with a valid token, no token, a malformed token, a token signed with the wrong secret, and a token whose subject had been deleted. Role authorisation was tested by presenting a user token to an administrative endpoint.

For the event module, creation was tested against valid input and against each boundary the schema enforces: zero tickets, a negative price, an over-length title, a missing required field. Slug generation was tested by creating two events with identical titles and confirming that distinct slugs resulted.

For the booking module the tests centred on the availability boundary: a quantity below availability, exactly equal to availability, and one above it. The exact-availability case is the important one, since it is where the sold-out transition fires.

For the payment module each channel was tested separately — free, demo gateway, and manual UPI — along with the double-confirmation case, in which an already-confirmed payment is confirmed a second time.

The value of testing at this level was that failures were unambiguous. When the over-availability booking initially succeeded, the fault could only lie in the availability check itself, because nothing else was involved.

5.3.2 Integration Testing

Integration testing verified that the modules behave correctly in combination, and that data crossing a boundary between them arrives intact.

The frontend-to-backend integration was tested by submitting each form through the interface and confirming that the request reached the correct endpoint with the expected payload and headers, and that the response was rendered correctly. The token interceptor was verified by confirming that a request issued after sign-in carried the `Authorization` header, and that one issued after sign-out did not.

The backend-to-database integration was tested by confirming that each write produced the expected document, that schema validators rejected invalid writes, and that the unique index on `email` rejected a duplicate registration even when the application-level check was bypassed by issuing the request directly.

The cross-module integrations were the ones that mattered most. The booking-to-event integration was verified by booking tickets and confirming that the event's available count decreased by exactly the quantity booked — and, on cancellation, increased by exactly the same amount and restored the published status. The booking-to-payment integration was verified by confirming that each booking produced exactly one payment record with the correct amount, provider and status, and that cancelling a paid booking marked its payment refunded. The administrative UPI confirmation flow was verified end to end: a customer submits a transfer reference, the payment is recorded as pending and the booking held, an administrator confirms it, and the booking moves to confirmed with the confirming administrator and timestamp recorded.

5.3.3 System Testing

System testing exercised the complete application in its deployed environment — the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas — rather than against local servers, so that deployment-specific behaviour was covered.

Two complete journeys were traced. In the user journey, a new visitor registers, signs in, browses the events listing, opens an event, books tickets, completes payment through each of the three channels in turn, views the booking in their history, downloads the QR-coded ticket as a PDF, cancels a booking and confirms that the tickets return to the event. In the administrator journey, an administrator signs in through the hidden route, reviews the

full event listing, features an event and confirms it appears on the landing page, reviews the booking list, confirms a pending UPI payment, verifies the ticket activates, transfers a booking to another account, and reviews the payment ledger and the user list.

Alongside these, cross-cutting behaviour was checked. Every protected route was requested without a token and with a token of the wrong role, and each was confirmed to reject the request. The interface was exercised at desktop, tablet and mobile viewport widths in both themes. The single-page routing was verified by deep-linking to and refreshing an interior route, which confirmed that the Vercel rewrite was serving `index.html` correctly. Cross-origin behaviour was verified by confirming that the API accepts requests from the deployed frontend origin and rejects them from another. The health endpoint was queried to confirm that it reports database connectivity accurately.

5.4 Modifications and Improvements

The following changes were made during development in response to problems found in testing or to weaknesses identified on review. They are presented in the order they occurred, which follows the project's version control history.

No.	Issue identified	Modification made
1	A page refresh on an interior route returned a 404 from the static host.	A rewrite rule was added in <code>vercel.json</code> directing all paths to <code>index.html</code> , so that client-side routing resolves the path after the shell loads.
2	Cancelling a booking on a sold-out event returned the tickets, but the event remained unbookable.	<code>restoreInventoryForBooking</code> was extended to reset the event's status from <code>sold_out</code> back to published whenever stock is returned.
3	Editing an event's venue or start time silently altered the displayed details of every past booking against it.	Booking and Payment schemas were given denormalised snapshot fields, captured at creation, so a historical record reflects the event as it stood at purchase.
4	The interface was usable on desktop but cramped on narrow viewports.	Layouts were reworked to Bootstrap's mobile-first breakpoints; listings collapse to a single column and administrative tables scroll within their container.
5	A single payment path was insufficient, since no live gateway was available.	A demo gateway and a manually-reviewed UPI channel were implemented side by side, both writing to a shared payment ledger.
6	Payment method strings were being derived independently in each payment controller, allowing them to drift.	The <code>methodByProvider</code> mapping and the <code>recordPayment</code> helper were extracted so that every channel writes ledger entries through one code path.
7	An administrative sign-in link in the public navigation advertised the existence of the administrative area.	The link was removed and the route made reachable through a hidden trigger — five rapid clicks on the footer text, or a keyboard shortcut.

No.	Issue identified	Modification made
8	Protected pages briefly redirected legitimate users to the sign-in screen while the stored session was being re-stored.	ProtectedRoute was given an explicit loading state, so a session still being restored renders a placeholder rather than being treated as unauthenticated.
9	Bundling the QR, canvas and PDF libraries made the initial payload large for every visitor, including those who never download a ticket.	The three libraries were moved behind dynamic <code>import()</code> calls executed at the moment a ticket is requested.
10	Administrators could see bookings but could not act on them when a customer needed assistance.	Administrative endpoints were added to create, cancel and transfer a booking on a user's behalf.
11	Featured events had to be unfeatured by hand once they had passed.	A <code>featuredUntil</code> expiry date was added alongside the <code>featured</code> flag.
12	There was no way to confirm from outside whether the API had reached the database.	A <code>/api/health</code> endpoint was added, reporting API and database status, and the connection module was reworked to track and expose its own state.

5.4.1 Outstanding Items

Two items identified during review remain outstanding and are recorded here for completeness. A diagnostic `console.log` of the configured API base URL remains in `src/services/api.js` and should be removed before any production release. A `PaymentOrder` model referring to a Razorpay order identifier remains in `backend/models/` from an abandoned attempt at commercial gateway integration; it is referenced by no route or controller and should be deleted. Neither affects the behaviour of the system, but both are the kind of residue that a reviewer will notice.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

6.1.1 Summary of Results

The forty test cases designed in Section 4.5 were executed against the deployed system. The table below summarises the outcome by module.

Module	Cases	Passed	Failed	Pass rate
User Management and Authentication	10	10	0	100%
Event Management	8	8	0	100%
Booking Management	8	8	0	100%
Payment Management	6	6	0	100%
Ticketing	2	2	0	100%
Administration	5	5	0	100%
System	1	1	0	100%
Total	40	40	0	100%

The figures above record the state of the system after the modifications described in Section 5.4. Several of these cases did not pass on first execution; the defects they exposed are the subject of the next section, and it is those results rather than the final totals that carry the useful information.

6.1.2 Defects Found and Resolved

Case	Observed behaviour	Cause	Resolution
TC-20	After a booking consumed the last available tickets, the event was marked sold out — but cancelling that booking returned the tickets and left the event still marked sold out and therefore unbookable.	The cancellation path restored the numeric count but did not reverse the status transition that the count had triggered.	The status reset was added to <code>restoreInventoryForBooking</code> , so any return of stock republishes a sold-out event.
TC-25	Cancelling a paid booking returned the tickets but left the associated payment recorded as paid.	The cancellation path updated the booking and the event but not the ledger.	<code>markPaymentsRefunded</code> was called as part of the cancellation, so the ledger reflects the reversal.
TC-17	Refreshing the browser on an event detail page returned a 404 from the static host.	The host had no rewrite rule, so it attempted to resolve the client-side route as a file path.	A catch-all rewrite to <code>index.html</code> was added in <code>vercel.json</code> .
TC-06	A signed-in user reloading a protected page was briefly redirected to the sign-in screen before being returned.	The route guard treated the interval during which the stored session was being restored as an unauthenticated state.	An explicit loading branch was added to <code>ProtectedRoute</code> .
TC-31	An already-confirmed UPI payment could be confirmed a second time, writing a second confirmation timestamp.	The confirmation handler did not check the payment's current status before acting.	A status guard was added, rejecting confirmation of a payment not in the pending state.
TC-14	Two events created with the same title produced a duplicate-key error on the slug index.	Slug derivation was a direct transformation of the title with no uniqueness check.	Slug generation was extended to test for collision and disambiguate.

6.1.3 Discussion

Two observations are worth drawing out of these results.

The defects clustered around state transitions, not around individual operations. Every operation worked correctly when tested on its own. What failed was the correspondence between operations — a status set by one action and not cleared by its inverse, a ledger updated on payment but not on refund, a confirmation that could be applied twice. This is the characteristic failure mode of a system holding related state in several places, and it is precisely why the availability invariant described in Section 4.2.2 was made explicit: once the rule is stated as "every decrement has a matching restore", the omissions become findable rather than a matter of luck.

Boundary cases were where the useful defects lived. Booking fewer tickets than were available worked from the beginning. Booking exactly the number available is what exposed both the sold-out reversal defect and, indirectly, the ledger defect. The general lesson — that the interesting case is the one at the edge of a condition rather than comfortably inside it — guided the later test design, and cases TC-20, TC-21 and TC-31 were all written specifically to sit on such boundaries.

The system's behaviour under the conditions tested is therefore sound, with the qualification that testing was functional rather than load-based. No load or stress testing was performed, so the system's behaviour under concurrent booking pressure — in particular, whether two simultaneous requests for the last remaining ticket could both succeed — has not been established empirically. This is recorded as a limitation in Section 7.2.

6.2 User Documentation

This section describes how to use the completed application. It is written for the two roles the system recognises, and assumes no technical knowledge.

6.2.1 Getting Started

The application is reached through a web browser at the deployed address and requires no installation. Visitors may browse published events and view their details without an account; registering is required only to book tickets or to publish an event.

6.2.2 For Registered Users

Creating an Account

Select **Sign up** from the navigation bar. Supply a name, an email address and a password of at least six characters, and confirm the password. The email address must not already be registered. On successful registration the account is created and signed in immediately.

Signing In

Select **Sign in** and supply the registered email address and password. On success the user dashboard is displayed. An incorrect email or password produces a message to that effect without indicating which of the two was wrong.

Browsing and Finding Events

The landing page presents a selection of featured events. The **Events** page lists every published event. Selecting an event opens its detail page, which shows the full description, category, organiser, venue and city, start and end date-times, ticket price and the number of tickets still available.

Booking Tickets

From an event's detail page, enter the number of tickets required and confirm the booking. The quantity must be at least one and must not exceed the tickets still available; a larger request is rejected with a message stating how many remain.

For a free event the booking is confirmed immediately. For a priced event the system proceeds to payment.

Paying for a Booking

Two payment channels are offered.

Demo gateway. Confirm the displayed amount. The payment is recorded and the booking confirmed immediately, and the ticket becomes available at once.

Manual UPI transfer. Transfer the displayed amount using the UPI details shown, then enter the transaction reference from the transfer into the form and submit it. The booking is held pending while an administrator verifies the transfer against the reference supplied. Once confirmed, the booking status changes to confirmed and the ticket becomes available. Until then the booking remains visible in the booking history with a pending status.

Viewing Bookings and Downloading Tickets

The **My Bookings** page lists every booking made by the account, showing the booking reference, the event details as they stood at the time of purchase, the quantity, the amount and the current status.

For a confirmed booking, opening the ticket displays it with a QR code encoding the booking reference. Selecting the download option saves the ticket as a PDF file, which can be printed or presented on a phone.

Cancelling a Booking

Select **Cancel** against a booking in the booking history and confirm. The booking's status becomes cancelled, the tickets are returned to the event and become available to others, and the associated payment is marked refunded. A booking already cancelled cannot be cancelled again.

Publishing an Event

Any registered user may publish an event. From **My Events**, select the option to create an event and complete the form: title, description, category, organiser name, venue, city, banner image, start and end date-times, ticket price and total tickets. The price may be zero, which makes the event free to book. Once published, the event appears in the public events

listing.

Events published by the account are listed on the **My Events** page, where they may be edited or removed. Bookings already made against an event retain the details in force at the time they were made, so editing an event does not alter existing bookings.

Profile and Appearance

A profile image may be set from the account menu. The theme toggle in the navigation bar switches between light and dark presentation, and the choice persists across visits.

Signing Out

Select **Sign out** from the account menu. The stored session is discarded and the public site is displayed.

6.2.3 For Administrators

Reaching the Administrative Sign-In

The administrative sign-in page is deliberately not linked from the public navigation. It is reached either by clicking the footer text on the left five times in quick succession, or by pressing Ctrl+Shift+L on any page. The page may also be opened directly at /admin/login.

The Administrative Dashboard

After signing in with administrative credentials, the dashboard gives access to four consoles: Events, Bookings, Payments and Users.

Managing Events

The Events console lists every event on the platform, including drafts and cancelled events, which the public listing excludes. Any event may be edited or deleted regardless of who published it. An event may be designated as featured, with an expiry date after which the designation lapses; featured events appear on the landing page.

Managing Bookings

The Bookings console lists every booking on the platform with its reference, the account that made it, the event, the quantity, the amount and its status. An administrator may create a booking on a user's behalf, cancel a booking — which returns the tickets to the event exactly as a user-initiated cancellation does — and transfer a booking to a different account, which is the appropriate action when a ticket changes hands.

Managing Payments

The Payments console shows the complete payment ledger: every attempt across all channels, with its reference, provider, method, amount, currency and status.

The console's principal function is the confirmation of manual UPI transfers. Payments with a pending status are awaiting review. For each, the customer's submitted transaction reference is shown; verify it against the receiving account's records, and if the transfer is genuine, confirm the payment. Confirming records the administrator's identity and the time, marks the payment paid, and confirms the associated booking so that the ticket activates.

Confirm a payment only after independently verifying that the transfer was received, since confirmation issues a valid ticket.

Managing Users

The Users console lists every registered account with its name, email, role and registration date. Password hashes are never included.

Checking System Health

The endpoint `/api/health` reports whether the API is responding and whether it has an active database connection. It is useful for confirming that the system is operational without signing in.

6.2.4 Troubleshooting

Situation	What to do
"An account with this email already exists" during registration	The address is already registered. Sign in instead, or register with a different address.

Situation	What to do
Sign-in is rejected although the credentials appear correct	Check for a trailing space in the email field. The address is not case-sensitive; the password is.
The booking is rejected as exceeding availability	Fewer tickets remain than requested. The message states how many are left.
A UPI booking still shows as pending	An administrator has not yet verified the transfer. The ticket activates once it is confirmed.
The ticket cannot be downloaded	Tickets are issued only for confirmed bookings. A booking awaiting payment or awaiting UPI confirmation has no ticket yet.
An event no longer appears in the listing	It may have been unpublished, cancelled, or sold out. Existing bookings against it are unaffected.
The session ends unexpectedly	Access tokens expire after a fixed period. Sign in again.

6.2.5 Interface Reference

[Section pending — annotated screenshots]

This section illustrates each step described above with an annotated screenshot of the corresponding screen: registration, sign-in, the events listing, an event detail page, the booking form, both payment screens, the booking history, the QR-coded ticket, the event creation form, and the four administrative consoles. Screenshots are to be inserted here.

CHAPTER 7

CONCLUSION

7.1 Conclusion

The objective set out at the beginning of this project was to replace the fragmented, manual administration of events with a single application in which an event, the tickets sold against it and the payments received for it form one connected record. That objective has been met. Eventify is a working, deployed, full-stack web application in which a registered user can publish an event, others can discover and book it, payment is settled through one of three channels and recorded in an auditable ledger, a QR-coded ticket is issued against every confirmed booking, and an administrator has oversight of the whole platform.

The technical foundation is a MERN stack: a React 18 single-page frontend bundled with Vite and styled with Bootstrap 5, an Express 5 REST API on Node.js, and a MongoDB Atlas cluster accessed through Mongoose. Authentication is stateless, with bcrypt-hashed passwords and signed JSON Web Tokens, and authorisation is enforced as composable route middleware so that role restrictions live in the route table rather than being scattered through controllers. The application was verified through forty designed test cases across unit, integration and system levels, and deployed with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas.

Three design decisions proved to be the ones that mattered.

Treating ticket inventory as a conserved quantity converted a class of administrative error into a structural impossibility. Because the same operation that writes a booking decrements the available count, and every cancellation restores it, the availability shown to a prospective attendee cannot drift from reality through the normal application flow. Overselling is not discouraged by the system; it is prevented by it.

Denormalising the event snapshot into bookings and payments solved a problem that was not obvious at the outset. A booking is a record of a transaction that occurred at a particular moment, and if it resolves the event by reference at read time, a later edit to that event silently rewrites history. Storing the title, slug, start date, venue and city as they stood

at purchase costs a small amount of duplication and buys a record that stays true.

Enforcing authorisation at the API rather than in the interface is the difference between a system that appears secure and one that is. Hiding an administrative link protects nothing; the route behind it is protected independently, and the interface guard exists for usability alone.

The defects surfaced during testing are also instructive, and they clustered in one place: not in individual operations, all of which worked in isolation, but in the correspondence between an operation and its inverse — a sold-out status set on booking and not cleared on cancellation, a ledger entry written on payment and not reversed on refund, a confirmation that could be applied twice. This is the characteristic failure mode of a system holding related state in more than one place, and identifying it explicitly made the remaining instances findable rather than a matter of chance.

Taken together, the project demonstrates that the operational problems of manual event administration — duplicated records, uncertain availability and untraceable payments — are addressable with a modest, carefully structured full-stack application, and it provides a foundation on which the extensions described in Section 7.3 can be built.

7.1.1 Significance of the System

The significance of the system lies in three areas.

Operationally, it removes the manual reconciliation that consumes most of the effort in running an event. Because registration, ticket count and payment are parts of one record rather than three artefacts kept in step by hand, the reconciliation step disappears entirely rather than merely becoming easier. For an organiser this is the difference between an administrative task and no task at all.

Structurally, it distributes event publication rather than centralising it. Because any registered user may publish an event and administrators moderate rather than originate, the platform suits a setting — a college in which individual societies each run their own programme, for instance — where a single administrative office would be a bottleneck. This is a deliberate departure from the conventional administrator-publishes model, and it changes what the platform is useful for.

Academically, the project demonstrates the practical construction of a complete full-stack application: component-based frontend architecture, RESTful API design with composed

middleware, schema modelling on a document database, stateless authentication with role-based authorisation, and independent deployment of the presentation and application tiers. The problems encountered — state transitions that were not reversed, records that changed retroactively, boundaries that were not tested — are the problems that characterise real systems rather than exercises, and working through them was the more valuable part of the exercise.

7.2 Limitations of the System

The system as delivered has the following limitations. They are stated plainly, since a clear account of what a system does not do is more useful than an optimistic one.

1. **Payment is not settled through a live gateway.** The demo gateway simulates settlement, and the UPI channel depends on an administrator manually verifying that a transfer was received. Neither produces the automatic, cryptographically verified confirmation that a commercial gateway provides, and the UPI channel introduces a human step between payment and ticket activation.
2. **Ticket QR codes are issued but not validated.** Every confirmed booking yields a scannable QR code, but no companion application exists to scan it at a venue entrance and mark it used. A ticket could therefore be presented more than once without the system detecting it.
3. **No notification of any kind is sent.** There is no email or SMS confirmation on registration, on booking, on payment, or when an event is changed or cancelled. A user learns of a change only by returning to the site.
4. **Concurrency under contention has not been established.** Booking performs its availability check and its inventory decrement as separate operations within one request rather than as a single atomic database operation. Under simultaneous requests for the last remaining ticket, the system has not been shown to reject one of them reliably. No load or stress testing was performed.
5. **Tokens cannot be revoked.** Because authentication is stateless, a token remains valid until it expires. There is no mechanism to invalidate a token early, so a compromised token stays usable for the remainder of its lifetime.
6. **Authentication endpoints are not rate limited.** The system does not itself slow repeated sign-in attempts, although bcrypt's cost factor makes automated guessing expensive.
7. **Administrative insight is limited to listings.** The consoles show every event, booking, payment and user, but provide no aggregation — no revenue totals, no sales trends, no measure of event performance.

8. **Search and filtering are basic.** Events can be browsed but not filtered by category, city, date range or price, which becomes a limitation as the number of listings grows.
9. **Feedback and rating functionality is specified but not implemented.** It is carried through the conceptual models and the modular design, and is scheduled for the next iteration.
10. **Seating is not modelled.** Tickets are counted, not assigned. Events at venues with numbered seats cannot be represented.
11. **Banner images are stored as base64 strings inside event documents** rather than in object storage, which inflates document size and is not an approach that scales to a large catalogue of events.
12. **There is no native mobile application.** The interface is responsive and works in a mobile browser, but offers none of the capabilities — offline ticket access, push notification — that a native application would.

7.3 Future Scope of the Project

Each of the following extends the system in a direction the current architecture already accommodates.

7.3.1 Live Payment Gateway Integration

Replacing the demo gateway with a commercial provider such as Razorpay or Stripe would give automatic, verified settlement and remove the manual confirmation step from the UPI path. The groundwork exists: the payment ledger already records a provider, a method and a transaction identifier for every attempt, so a new provider is an additional entry in the existing enumeration rather than a restructuring. Webhook handling would be required so that settlement confirmation arrives from the provider rather than from the client.

7.3.2 Ticket Validation at the Venue

A companion scanning application — a web application usable from any phone camera would suffice — would read a ticket’s QR code, verify the booking against the API and mark it as used. This closes the gap identified in Section 7.2: a ticket presented a second time would be rejected. It requires a scanned-at timestamp on the booking and one additional endpoint.

7.3.3 Notification

Transactional email and SMS on registration, booking, payment confirmation, event modification and event cancellation, together with a reminder before an event begins. Since the system already records every state transition, notification is a matter of dispatching a message at each transition rather than of tracking new state.

7.3.4 Atomic Inventory Operations

The concurrency limitation would be addressed by replacing the check-then-decrement pattern with a single conditional update — a `findOneAndUpdate` that decrements the available count only if it is at least the requested quantity, and fails otherwise. This makes the check and the decrement one indivisible database operation, so two simultaneous requests for a last remaining ticket cannot both succeed.

7.3.5 Token Revocation and Rate Limiting

Introducing refresh tokens with short-lived access tokens, backed by a revocation list, would allow a compromised session to be terminated without abandoning the benefits of stateless authentication. Rate limiting on the authentication endpoints would slow automated credential guessing at the perimeter.

7.3.6 Search, Filtering and Recommendation

Filtering by category, city, date range, price and availability, with full-text search over titles and descriptions — both supported natively by MongoDB. Beyond that, a recommendation mechanism suggesting events based on a user's booking history and the categories they browse.

7.3.7 Analytics and Reporting

An administrative dashboard aggregating ticket sales, revenue, booking trends over time, event popularity by category and city, and cancellation rates, with export to spreadsheet for offline analysis. MongoDB's aggregation pipeline supports this directly.

7.3.8 Seat Selection

A visual seat map for venues with assigned seating, allowing a customer to choose specific seats before confirming. This requires extending the event model with a seating layout and the booking model with the specific seats reserved.

7.3.9 Feedback and Rating

Implementation of the module specified in Section 4.1.7: a rating and comment submitted by an attendee holding a confirmed booking, aggregated into an average rating displayed on the event, and reviewable by administrators.

7.3.10 Media Storage

Moving banner images out of event documents into an object store, with the document retaining only a URL. This reduces document size, allows images to be served from a

content delivery network, and removes a scaling constraint.

7.3.11 Native Mobile Applications

Android and iOS applications consuming the same REST API, offering offline access to purchased tickets and push notification — the two capabilities a responsive web interface cannot provide.

7.3.12 Multi-Language Support

Interface localisation, which broadens the platform's reach in a multilingual setting.

These extensions are directions in which the system could develop rather than commitments within the scope of the present academic project. What matters for the work described in this report is that none of them requires the existing architecture to be dismantled: each is an addition to a structure that was designed to accommodate it.

BIBLIOGRAPHY

The following sources were consulted during the design, implementation and testing of the Smart Event Management and Ticket Booking System. All online sources were last accessed on 27 August 2026.

Official Documentation

- [1] Meta Open Source. *React Documentation — Learn React*. Available at: <https://react.dev/learn>
- [2] Meta Open Source. *React Documentation — Passing Data Deeply with Context*. Available at: <https://react.dev/learn/passing-data-deeply-with-context>
- [3] Vite. *Vite Guide — Getting Started*. Available at: <https://vite.dev/guide/>
- [4] Vite. *Vite Guide — Building for Production*. Available at: <https://vite.dev/guide/build>
- [5] Remix Software. *React Router Documentation*. Available at: <https://reactrouter.com/>
- [6] OpenJS Foundation. *Node.js Documentation — API Reference*. Available at: <https://nodejs.org/docs/latest/api/>
- [7] OpenJS Foundation. *Express 5.x — API Reference*. Available at: <https://expressjs.com/en/5x/api.html>
- [8] OpenJS Foundation. *Express — Using Middleware*. Available at: <https://expressjs.com/en/guide/using-middleware.html>
- [9] MongoDB, Inc. *MongoDB Manual — Data Modeling*. Available at: <https://www.mongodb.com/docs/manual/data-modeling/>
- [10] MongoDB, Inc. *MongoDB Manual — Indexes*. Available at: <https://www.mongodb.com/docs/manual/indexes/>
- [11] MongoDB, Inc. *MongoDB Atlas Documentation*. Available at: <https://www.mongodb.com/docs/atlas/>

com/docs/atlas/

- [12] Automattic. *Mongoose Documentation — Guide*. Available at: <https://mongoosejs.com/docs/guide.html>
- [13] Automattic. *Mongoose Documentation — Validation*. Available at: <https://mongoosejs.com/docs/validation.html>
- [14] Automattic. *Mongoose Documentation — Middleware*. Available at: <https://mongoosejs.com/docs/middleware.html>
- [15] Bootstrap Team. *Bootstrap 5 Documentation — Layout and Grid System*. Available at: <https://getbootstrap.com/docs/5.3/layout/grid/>
- [16] Axios. *Axios Documentation — Interceptors*. Available at: <https://axios-http.com/docs/interceptors>
- [17] MDN Web Docs. *HTTP — Cross-Origin Resource Sharing (CORS)*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- [18] MDN Web Docs. *JavaScript — Modules and Dynamic Imports*. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- [19] MDN Web Docs. *CSS — Using CSS Custom Properties*. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- [20] Vercel. *Vercel Documentation — Project Configuration*. Available at: <https://vercel.com/docs/project-configuration>
- [21] Railway. *Railway Documentation — Deploying a Node.js Application*. Available at: <https://docs.railway.com/>
- [22] Git. *Pro Git Book*, 2nd edition. Available at: <https://git-scm.com/book/en/v2>
- [23] Postman. *Postman Learning Center — Testing an API*. Available at: <https://learning.postman.com/docs/>
- [24] MKLab. *StarUML Documentation*. Available at: <https://docs.staruml.io/>

Standards and Specifications

- [25] Jones, M., Bradley, J. and Sakimura, N. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force, 2015. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
- [26] Fielding, R. and Reschke, J. *RFC 7231: Hypertext Transfer Protocol (HTTP/1.1) — Semantics and Content*. Internet Engineering Task Force, 2014. Available at: <https://datatracker.ietf.org/doc/html/rfc7231>
- [27] Provos, N. and Mazières, D. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference, 1999.
- [28] Object Management Group. *OMG Unified Modeling Language (OMG UML)*, Version 2.5.1. Available at: <https://www.omg.org/spec/UML/>
- [29] International Organization for Standardization. *ISO/IEC 18004: Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*.

Security Guidance

- [30] OWASP Foundation. *OWASP Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [31] OWASP Foundation. *OWASP Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [32] OWASP Foundation. *OWASP Top Ten Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/>

Books

- [33] Flanagan, D. *JavaScript: The Definitive Guide*, 7th edition. O'Reilly Media, 2020.
- [34] Chodorow, K. *MongoDB: The Definitive Guide*, 3rd edition. O'Reilly Media, 2019.
- [35] Sommerville, I. *Software Engineering*, 10th edition. Pearson, 2015.

- [36] Pressman, R. S. and Maxim, B. R. *Software Engineering: A Practitioner's Approach*, 8th edition. McGraw-Hill Education, 2014.
- [37] Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

GLOSSARY

Term	Definition
API	Application Programming Interface. The set of endpoints through which the frontend communicates with the backend.
Axios	A JavaScript HTTP client used by the frontend to call the API.
bcrypt	A deliberately slow password hashing function with a configurable cost factor, used here with ten salt rounds.
BSON	Binary JSON. The binary encoding MongoDB uses to store documents.
Bootstrap	A CSS framework providing a responsive grid and pre-built interface components.
Collection	In MongoDB, a group of documents; the document-database counterpart of a relational table.
CORS	Cross-Origin Resource Sharing. The browser mechanism that controls which origins may issue requests to a server.
Controller	A backend module holding the request handling and business logic for one area of the system.
Denormalisation	Deliberately duplicating data across documents to avoid resolving references at read time — used here so that bookings preserve the event as it stood at purchase.
Document	In MongoDB, a single record, structured as a set of field-value pairs.
Enum validator	A Mongoose constraint restricting a field to a fixed set of permitted values.
Express	The web application framework layered over Node.js, providing routing and a middleware pipeline.
Hashing	A one-way transformation of data into a fixed-length value that cannot practically be reversed.
Hot module replacement	A development feature that applies source edits to a running page without a full reload.
JSON	JavaScript Object Notation. The text format used for data exchange between the tiers.

Term	Definition
JWT	JSON Web Token. A signed token carrying identity claims, used here to authenticate requests without server-side sessions.
MERN	The stack composed of MongoDB, Express, React and Node.js.
Middleware	A function in the Express pipeline that inspects or modifies a request before it reaches a controller.
MongoDB Atlas	The managed cloud service on which the project's MongoDB cluster runs.
Mongoose	The object data modelling library that adds schema definition and validation above MongoDB.
Node.js	The server-side JavaScript runtime on which the backend executes.
NoSQL	A category of databases that do not use the relational model; MongoDB is a document-oriented example.
ObjectId	The twelve-byte identifier MongoDB assigns to every document as its primary key.
ODM	Object Data Modelling. The layer, here Mongoose, that maps application objects to database documents.
Populate	Mongoose's mechanism for resolving a reference into the referenced document — the document-database counterpart of a join.
QR code	A two-dimensional barcode; used here to encode a booking reference on a ticket.
React	The JavaScript library used to build the frontend from composable components.
REST	Representational State Transfer. The architectural style the API follows, addressing resources by URL and acting on them with HTTP methods.
Role-based access control	Restricting operations according to the role held by the authenticated account.
Salt	Random data incorporated into a password hash so that identical passwords produce different stored values.
Schema	In Mongoose, the declaration of a collection's permitted fields, their types and their constraints.
Slug	A URL-friendly identifier derived from an event's title.
SPA	Single-Page Application. A web application that loads once and updates the view through client-side routing.

Term	Definition
UPI	Unified Payments Interface. The Indian real-time payment system used by the project's manual payment channel.
Vercel	The hosting platform serving the frontend bundle.
Railway	The hosting platform running the backend service.
Vite	The build tool and development server used for the frontend.
Virtual DOM	React's in-memory representation of the interface, diffed between renders to compute the minimum set of updates.